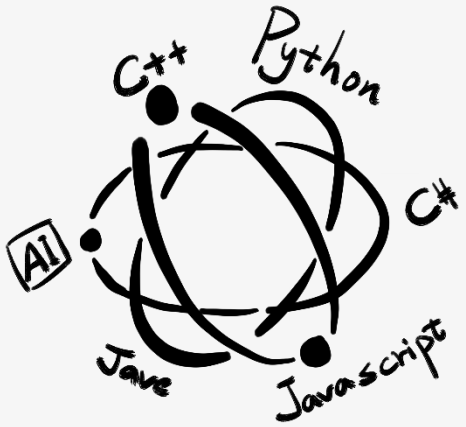




字符串

酷哥创客 COOLMAKER







1 温故知新

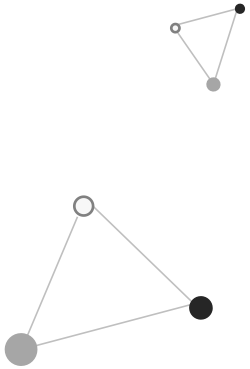
2 知识宝典

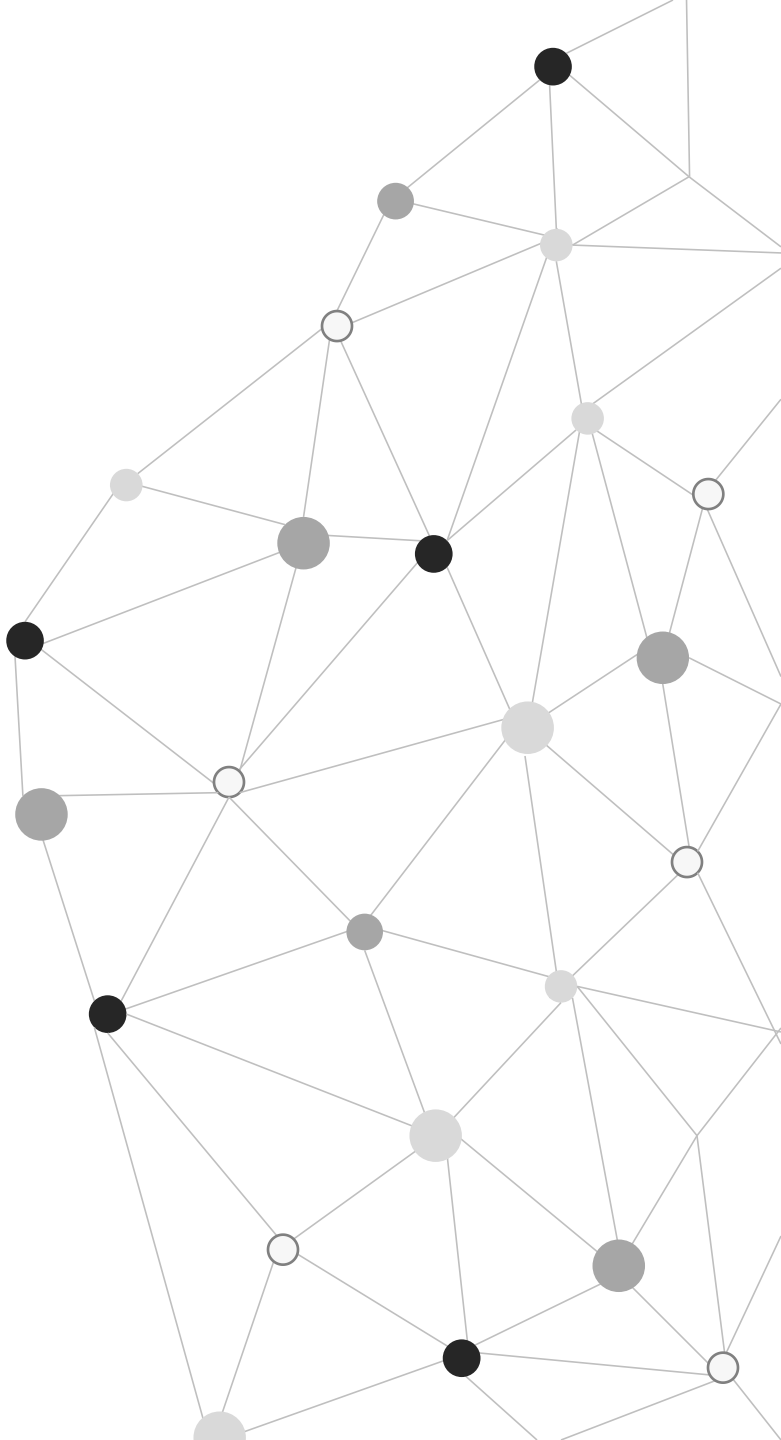
3 运用自如

4 回顾总结



知 识 宝 典







COOLMAKER
酷哥创客 人工智能教育



字符串



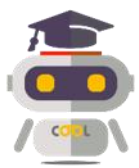


5.字符串算法

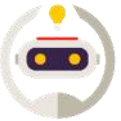
- 【6】字符串匹配：KMP算法
- 【7】Manacher算法
- 【6】字典树（Trie）

5.哈希表

- 【5】数值哈希函数构造
- 【6】字符串哈希函数构造
- 【6】哈希冲突的常用处理方法







COOLMAKER
酷哥创客 人工智能教育

CSP-J的变化:

●指针类型中：新增了5级知识点【引用】

●新增算法策略内含：

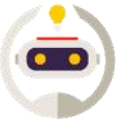
3级知识点--【前缀和】

4级知识点--【差分】

●初等数论中，从23年的【模运算与同余】修改为了【模运算与取余】

●删除2级知识点【格雷码】





COOLMAKER
酷哥创客 人工智能教育

CSP-S的变化:

●STL模版：新增了【位集合bitset】

●算法策略：新增了7级知识点【扫描线】

●字符串算法：新增了7级知识点【Manacher算法】(从NOI级别转移而来)

●图论算法：删除了7级知识点【次小生成树】

●动态规划：新增了6级知识点【多维动态规划】



Manacher算法





Manacher算法

COOLMAKER
酷哥创客 人工智能教育

描述

给定一个长度为 n 的字符串 s，请找到所有对 (i, j) 使得子串 $s[i \dots j]$ 为一个回文串。当 $t = t_{\text{rev}}$ 时，字符串 t 是一个回文串（t_{rev} 是 t 的反转字符串）。

显然在最坏情况下可能有 $O(n^2)$ 个回文串，因此似乎一眼看过去该问题并没有线性算法。

但是关于回文串的信息可用 **一种更紧凑的方式** 表达：对于每个位置 $i = 0 \dots n-1$，我们找出值 $d_1[i]$ 和 $d_2[i]$。二者分别表示以位置 i 为中心的长度为奇数和长度为偶数的回文串个数。换个角度，二者也表示了以位置 i 为中心的最长回文串的半径长度（半径长度 $d_1[i]$，$d_2[i]$ 均为从位置 i 到回文串最右端位置包含的字符个数）。



Manacher算法

COOLMAKER
酷哥创客 人工智能教育

举例来说，字符串 $s = \texttt{abababc}$ 以 $s[3] = b$ 为中心有三个奇数长度的回文串，最长回文串半径为 3，也即 $d_1[3] = 3$：

$$a\,\overbrace{b\,a\,b\,a\,b}^{d_1[3]=3}\,c$$
$$s_3$$

字符串 $s = \texttt{cbaabd}$ 以 $s[3] = a$ 为中心有两个偶数长度的回文串，最长回文串半径为 2，也即 $d_2[3] = 2$：

$$c\,\overbrace{b\,a\,a\,b}^{d_2[3]=2}\,d$$
$$s_3$$



Manacher算法

COOLMAKER
酷哥创客 人工智能教育

因此关键思路是，如果以某个位置 i 为中心，我们有一个长度为 l 的回文串，那么我们有以 i 为中心的长度为 $l-2$，$l-4$，等等的回文串。所以 $d_1[i]$ 和 $d_2[i]$ 两个数组已经足够表示字符串中所有子回文串的信息。



Manacher算法

COOLMAKER
酷哥创客 人工智能教育

朴素算法

该算法通过下述方式工作：对每个中心位置 i，在比较一对对应字符后，只要可能，该算法便尝试将答案加 1。

该算法是比较慢的：它只能在 $O(n^2)$ 的时间内计算答案。

```
vector<int> d1(n), d2(n);
for(int i = 0; i < n; i++)
{
    d1[i] = 1;
    while(0 <= i - d1[i] && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]])
    {
        d1[i]++;
    }
    d2[i] = 0;
    while(0 <= i - d2[i] - 1 && i + d2[i] < n && s[i - d2[i] - 1] == s[i + d2[i]])
    {
        d2[i]++;
    }
}
```



COOLMAKER
酷哥创客 人工智能教育

解法

总的来说，该问题具有多种解法：应用字符串哈希，该问题可在 $O(n \log n)$ 时间内解决，而使用后缀数组和快速 LCA 该问题可在 $O(n)$ 时间内解决。





对于一个给定的字符串str，下标从0开始，可以先求其字符串hash数组Hash1，然后将str翻转，求出翻转字符串str的hash数组Hash2，接着分回文串的奇偶情况讨论：

回文串的长度是奇数：

- 枚举回文中心i，二分子串的半径k，找到最大使子串[i-k,i+k]是回文串的k。
- 其中判断子串[i-k,i+k]是回文串等价于判断str的两个子串[i-k,i]与[i,i+k]是否是相反的串；而这等价于判断str[i-k,i]子串与反转字符串rstr的[len - 1-(i+k),len - 1 - i]子串是否是相同的
- [a,b]在反转字符串中的位置为[len - 1 -b, len -1 - a]

回文串的长度是偶数：

- 枚举回文空隙点，令i表示空隙点左边第一个元素的下标，二分子串的半径k，找到最大使子串[i-k+1,i+k]是回文串的k。
- 其中判断子串[i-k+1,i+k]是回文串等价于判断str的两个子串[i-k+1,i]与[i+1,i+k]是否是相反的串；而这等价与判断str的[i-k+1,i]子串与反字符串rstr的[len - 1 - (i+k),len - 1 - (i + 1)]子串是否是相同的

```cpp
void init()
{
    b_pow[0]=1,Hash1[0]=0,Hash2[0]=0;
    n=s.length();
    for(int i=1;i<=n;i++)
    {
        //计算base的i次方
        b_pow[i]=b_pow[i-1]*base%mod;
        //预处理部分hash函数值，hs i代表 0~i-1的字符串hash值
        Hash1[i]=(Hash1[i-1]*base+s[i-1])%mod;
        Hash2[i]=(Hash2[i-1]*base+s[n-i])%mod;//关键部分求反串hash
    }
}
```

```cpp
//求奇回文最大长度
int ans=1;
for(int i=0;i<n;i++)
{
    int l=1,r=min(i,n-1-i),k=0;
    while(l<=r)
    {
        int mid=(l+r)/2;
        if(get_hash1(i-mid,i)==get_hash2(n-1-(i+mid),n-1-i))////比较[i-mid,i]和[i,i+mid]
        {
            l=mid+1;
            k=mid;
        }else r=mid-1;
    }
    ans=max(ans,k*2+1);
}
```

```cpp
//求偶回文最大长度
for(int i=0;i<n;i++)
{
    int l=1,r=min(i+1,n-1-i),k=0;
    while(l<=r)
    {
        int mid=(l+r)/2;
        if(get_hash1(i-mid+1,i)==get_hash2(n-1-(i+mid),n-1-(i+1)))//比较[i-mid+1,i]和[i+1,i+mid]
        {
            l=mid+1;
            k=mid;
        }else r=mid-1;
    }
    ans=max(ans,k*2);
}
```

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int base=131;
const int mod=1e9+7;
string s;
int n;
ll b_pow[1000005],Hash1[1000005],Hash2[1000005];
void init()
{...}
ll get_hash1(int l,int r)
{
    return (Hash1[r+1]-(Hash1[l]*b_pow[r-l+1])%mod+mod)%mod;
}
ll get_hash2(int l,int r)
{
    return (Hash2[r+1]-(Hash2[l]*b_pow[r-l+1])%mod+mod)%mod;
}
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    for(int tn=1;cin>>s;tn++)
    {
        if(s=="END")break;
        init();
        /*
         abcbak   kabcba   i= 0 1 2 3 4 5      原位置
         012345   012345  ri=n-1 n-2 n-3 n-4 n-5 n-6  反串位置
        */
        //求奇回文最大长度
        int ans=1;
        for(int i=0;i<n;i++)
        {...}
        //求偶回文最大长度
        for(int i=0;i<n;i++)
        {...}
        cout<<"Case "<<tn<<": "<<ans<<"\n";
    }
    return 0;
}
```

Manacher算法

COOLMAKER
酷哥创客 人工智能教育

解法

总的来说，该问题具有多种解法：应用字符串哈希，该问题可在 $O(n \log n)$ 时间内解决，而使用后缀数组和快速 LCA 该问题可在 $O(n)$ 时间内解决。

但是这里描述的算法 **压倒性** 的简单，并且在时间和空间复杂度上具有更小的常数。该算法由 **Glenn K. Manacher** 在 1975 年提出。



Manacher算法

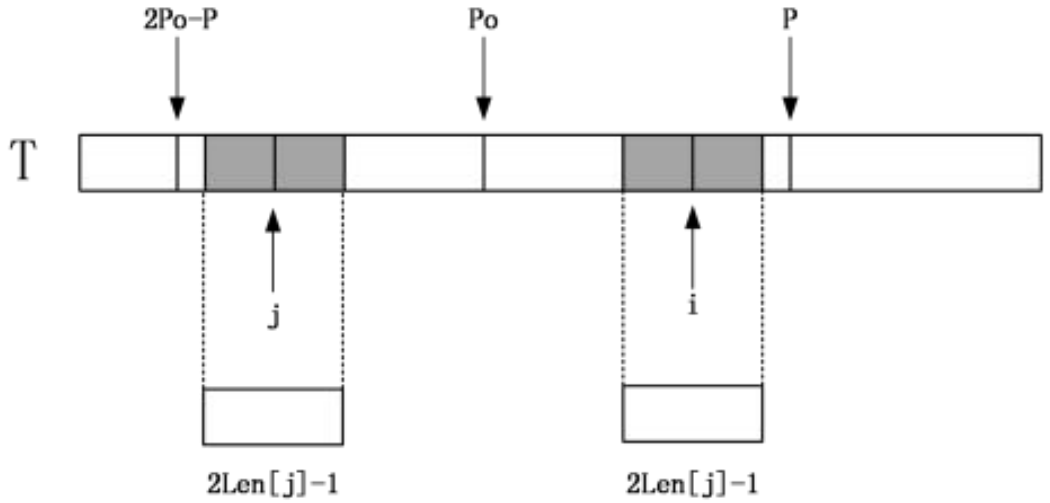
COOLMAKER
酷哥创客 人工智能教育

核心性质：

在暴力做法中维护两个半径数组，Manacher算法也是一样去维护这**两个半径数组**，特别的，我们可以利用一些规律去优化推导半径数组。

求关于下标 i 的半径数组，如果 i 在一个回文串的内部，回文中心为 po，那么可以在这个回文串中找到对称位置 j，所以**以 i 为中心**的回文串的长度至少和**以 j 为中心**的回文串在**以 po 为中心**的回文串中对称的，一样的，即 **Len[i] = Len[j]**。

如果 **Len[j] 超过 po 回文串右边界**，由对称性，说明以 i 为中心的回文串可能会延伸到之外，而大于右边界的部分我们还没有进行匹配，所以要从右边界开始一个一个暴力进行匹配，直到发生失配。



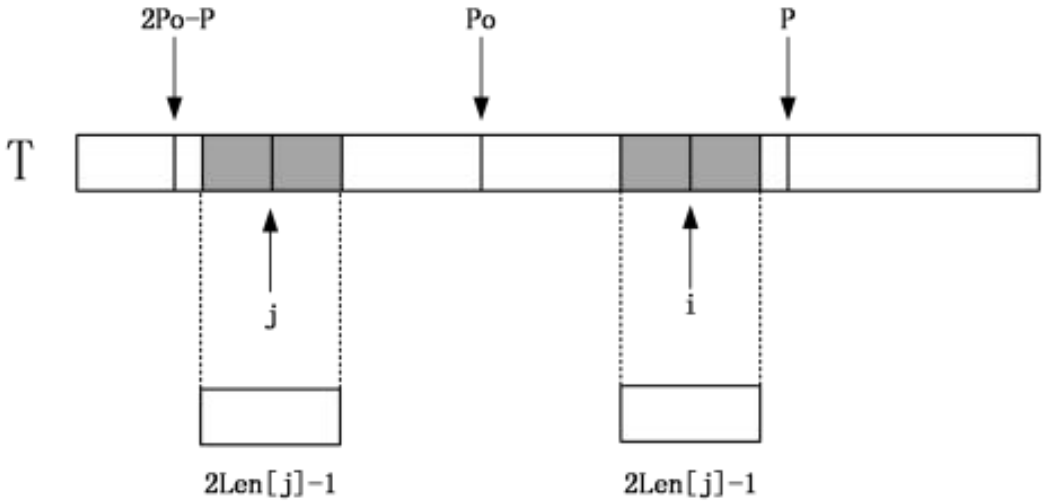
Manacher算法

COOLMAKER
酷哥创客 人工智能教育

核心性质：

在暴力做法中维护两个半径数组，Manacher算法也是一样去维护这两个半径数组，特别的，我们可以利用一些规律去优化推导半径数组。

如果 i **不在之前任何一个回文串的内部**，说明对于**中点为 i 的回文串**还一点都没有匹配，这个时候，就只能老老实实地一个一个暴力匹配了。



COOLMAKER
酷哥创客 人工智能教育

Manacher 算法

这里我们将只描述算法中寻找所有奇数长度子回文串的情况，即只计算 $d_1[]$；寻找所有偶数长度子回文串的算法（即计算数组 $d_2[]$）将只需对奇数情况下的算法进行一些小修改。

为了快速计算，我们维护已找到的最靠右的子回文串的 **边界** (l, r)（即具有最大 r 值的回文串，其中 l 和 r 分别为该回文串左右边界的位置）。初始时，我们置 $l = 0$ 和 $r = -1$（-1需区别于倒序索引位置，这里可为任意负数，仅为了循环初始时方便）。



COOLMAKER
酷哥创客 人工智能教育

①当前点在最远回文串外部，尝试暴力匹配

B	A	A	A	A	A	B	C
1	?	?	?	?	?	?	?

②对称过去在回文串内部可以直接对称过去

B	A	A	A	A	A	B	C
1	1	2	4	?	?	?	?

③对称过去恰好到最远回文串边界，回文串外面的可以尝试暴力匹配

B	A	A	A	A	A	B	A	A
1	1	2	4	2	1	?	?	?

④对称过去会超过最远回文串的边界，保留不超过的部分，对超出部分的对称性无法保证，尝试暴力匹配

A	A	B	A	A	A	A	A	B	A	C
1	1	3	1	2	5	2	1	?	?	?

⑤每次暴力匹配成功都意味着出现了一个最远的回文串，更新最远回文串

Manacher算法

COOLMAKER
酷哥创客 人工智能教育

现在假设我们要对下一个 i 计算 $d_1[i]$，而之前所有 $d_1[]$ 中的值已计算完毕。我们将通过下列方式计算：

- 如果 i 位于当前子回文串之外，即 $i > r$，那么我们调用朴素算法。

```
while(0 <= i - k && i + k < n && s[i - k] == s[i + k]) k++;
```

因此我们将连续地增加 $d_1[i]$，同时在每一步中检查当前的子串 $[i - d_1[i] \dots i + d_1[i]]$（$d_1[i]$ 表示半径长度，下同）是否为一个回文串。如果我们找到了第一处对应字符不同，又或者碰到了 s 的边界，则算法停止。在两种情况下我们均已计算完 $d_1[i]$。此后，仍需记得更新 (l, r)。

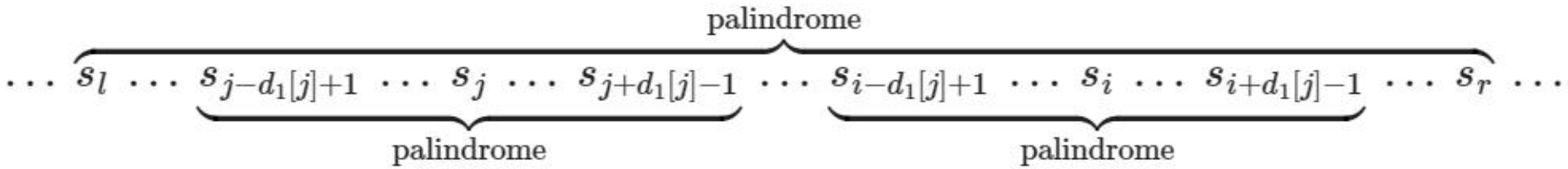
```
d1[i] = k--;
if(i + k > r)
{
    l = i - k;
    r = i + k;
}
```



Manacher算法

COOLMAKER
酷哥创客 人工智能教育

- 现在考虑 $i \le r$ 的情况。我们将尝试从已计算过的 $d_1[]$ 的值中获取一些信息。首先在子回文串 (l, r) 中反转位置 i，即我们得到 $j = l + (r - i)$。现在来考察值 $d_1[j]$。因为位置 j 同位置 i 对称，我们 **几乎总是** 可以置 $d_1[i] = d_1[j]$。该想法的图示如下（可认为以 j 为中心的回文串被「拷贝」至以 i 为中心的位置上）：



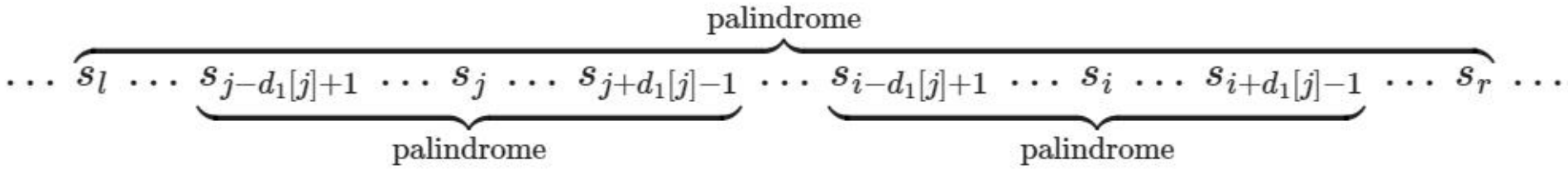
可以直接对称过去

B	A	A	A	A	A	B	C
1	1	2	4	?	?	?	?

Manacher算法

COOLMAKER
酷哥创客 人工智能教育

- 现在考虑 $i \le r$ 的情况。我们将尝试从已计算过的 $d_1[]$ 的值中获取一些信息。首先在子回文串 (l, r) 中反转位置 i，即我们得到 $j = l + (r - i)$。现在来考察值 $d_1[j]$。因为位置 j 同位置 i 对称，我们 **几乎总是** 可以置 $d_1[i] = d_1[j]$。该想法的图示如下（可认为以 j 为中心的回文串被「拷贝」至以 i 为中心的位置上）：




```
int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
```

对称过去会超过，保留不超过的部分，超出的暴力匹配

A	B	A	A	A	A	A	B
1	2	1	2	4	2	1	?

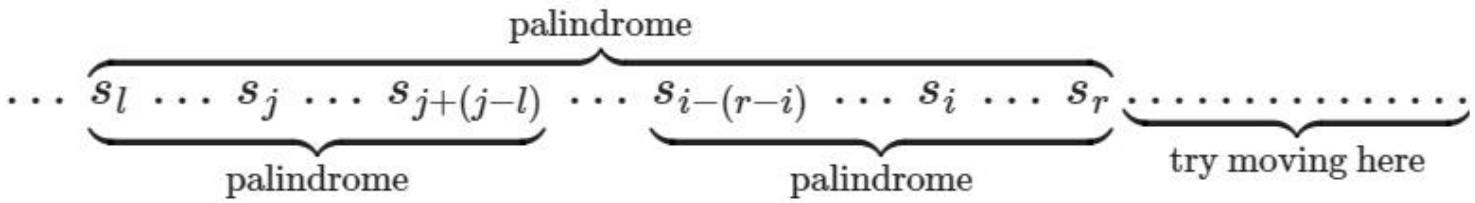
Manacher算法

COOLMAKER
酷哥创客 人工智能教育

然而有一个 **棘手的情况** 需要被正确处理：当「内部」的回文串到达「外部」回文串的边界时，即 $j - d_1[j] + 1 \le l$（或者等价的说，$i + d_1[j] - 1 \ge r$）。因为在「外部」回文串范围以外的对称性没有保证，因此直接置 $d_1[i] = d_1[j]$ 将是不正确的：我们没有足够的信息来断言在位置 i 的回文串具有同样的长度。

实际上，为了正确处理这种情况，我们应该「截断」回文串的长度，即置 $d_1[i] = r - i$。之后我们将运行朴素算法以尝试尽可能增加 $d_1[i]$ 的值。

该种情况的图示如下（以 j 为中心的回文串已经被截断以落在「外部」回文串内）：



B	A	A	A	A	A	B	A	A
1	1	2	4	2	1	?	?	?

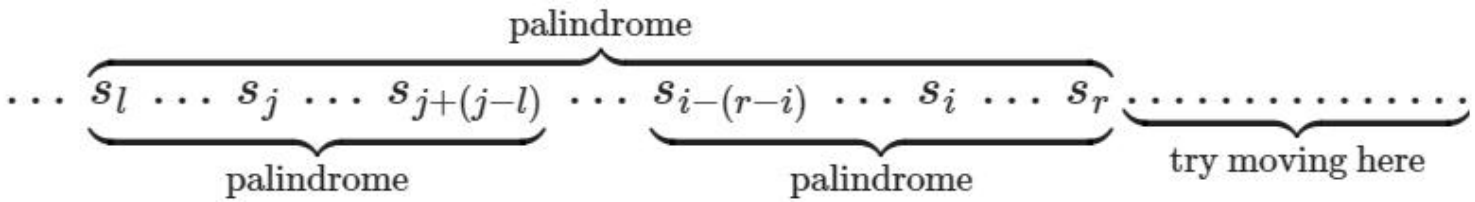
Manacher算法

COOLMAKER
酷哥创客 人工智能教育

然而有一个 **棘手的情况** 需要被正确处理：当「内部」的回文串到达「外部」回文串的边界时，即 $j - d_1[j] + 1 \leq l$（或者等价的说，$i + d_1[j] - 1 \geq r$）。因为在「外部」回文串范围以外的对称性没有保证，因此直接置 $d_1[i] = d_1[j]$ 将是不正确的：我们没有足够的信息来断言在位置 i 的回文串具有同样的长度。

实际上，为了正确处理这种情况，我们应该「截断」回文串的长度，即置 $d_1[i] = r - i$。之后我们将运行朴素算法以尝试尽可能增加 $d_1[i]$ 的值。

该种情况的图示如下（以 j 为中心的回文串已经被截断以落在「外部」回文串内）：

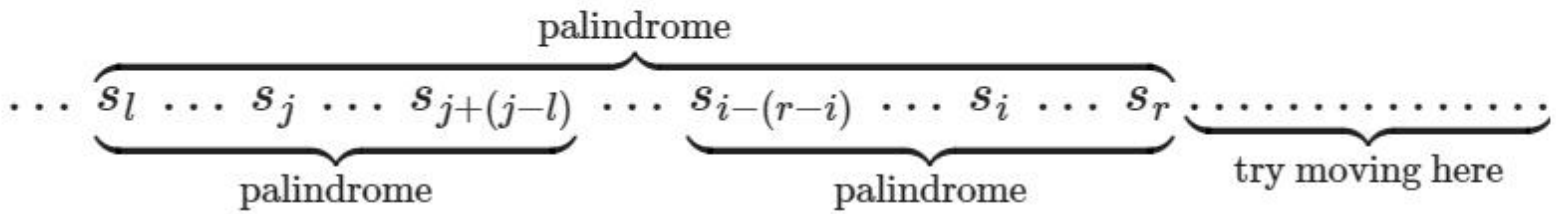



```
int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
while(0 <= i - k && i + k < n && s[i - k] == s[i + k]) k++;
```

Manacher算法

COOLMAKER
酷哥创客 人工智能教育

该种情况的图示如下（以 j 为中心的回文串已经被截断以落在「外部」回文串内）：



该图示显示出，尽管以 j 为中心的回文串可能更长，以致于超出「外部」回文串，但在位置 i，我们只能利用其完全落在「外部」回文串内的部分。然而位置 i 的答案可能比这个值更大，因此接下来我们将运行朴素算法来尝试将其扩展至「外部」回文串之外，也即标识为 "try moving here" 的区域。

```
int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
while(0 <= i - k && i + k < n && s[i - k] == s[i + k]) k++;
```

计算偶数长度回文串数组 $d_2[]$ 的算法同上述计算奇数长度回文串

Manacher算法

COOLMAKER
酷哥创客 人工智能教育

Manacher 算法的实现

分类讨论

计算 $d_1[]$

```
vector<int> d1(n);
for(int i = 0, l = 0, r = -1; i < n; i++)
{
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
    while(0 <= i - k && i + k < n && s[i - k] == s[i + k])
    {
        k++;
    }
    d1[i] = k--;
    if(i + k > r)
    {
        l = i - k;
        r = i + k;
    }
}
```



Manacher算法

COOLMAKER
酷哥创客 人工智能教育

计算 $d_2[]$ 的代码十分类似，但是在算术表达式上有些许不同：

```
vector<int> d2(n);
for(int i = 0, l = 0, r = -1; i < n; i++)
{
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while(0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k])
    {
        k++;
    }
    d2[i] = k--;
    if(i + k > r)
    {
        l = i - k - 1;
        r = i + k;
    }
}
```



Manacher算法模板

```
vector<int> d1(n), d2(n);
for(int i = 0; i < n; i++)
{
    d1[i] = 1;
    while(0 <= i - d1[i] && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]])
    {
        d1[i]++;
    }
    d2[i] = 0;
    while(0 <= i - d2[i] - 1 && i + d2[i] < n && s[i - d2[i] - 1] == s[i + d2[i]])
    {
        d2[i]++;
    }
}
```

```
struct Manacher
{
    int n,ans=0;
    string s;
    int manacher()
    {
        n=s.size();
        vector<int> d1(n),d2(n);
        for(int i = 0, l = 0, r = -1, l2 = 0, r2 = -1; i < n; i++)
        {
            //d1[i]: 以i为中心奇数长度最长回文串半径，长度为k*2-1
            int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
            while(0 <= i - k && i + k < n && s[i - k] == s[i + k]) k++;
            d1[i] = k--;
            if(i + k > r)
            {
                l = i - k;
                r = i + k;
            }
            //d2[i]: 以i-1,i为中心偶数长度最长回文串半径，长度为k*2
            k = (i > r2) ? 0 : min(d2[l2 + r2 - i + 1], r2 - i + 1);
            while(0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) k++;
            d2[i] = k--;
            if(i + k > r2)
            {
                l2 = i - k - 1;
                r2 = i + k;
            }
            ans=max({ans,d1[i]*2-1,d2[i]*2});
        }
        return ans;
    }
}mnc;
```

统一处理

虽然在讲解过程及上述实现中我们将 $d_1[]$ 和 $d_2[]$ 的计算分开考虑，但实际上可以通过一个技巧将二者的计算统一为 $d_1[]$ 的计算。

给定一个长度为 n 的字符串 s，我们在其 $n+1$ 个空中插入分隔符 $\#$，从而构造一个长度为 $2n+1$ 的字符串 s'。举例来说，对于字符串 $s = \texttt{ababab}$，其对应的 $s' = \texttt{\#a\#b\#a\#b\#a\#b\#c\#}$。

对于字母间的 $\#$，其实际意义为 s 中对应的「空」。而两端的 $\#$ 则是为了实现的方便。

```cpp
struct Manacher
{
    int n;
    char s[maxn], t[maxn << 1];
    int p[maxn << 1];
    int manacher()
    {
        n = strlen(s + 1);
        t[0] = '^', t[1] = '#';; // 扩展原字符串
        for(int i = 1; i <= n; i++)
        {
            t[i << 1] += s[i];
            t[i << 1 | 1] = '#';
        }
        t[(n << 1) + 2] = '$';
        n = n << 1;
        int ans = 0;
        for(int i = 2, ml = 0, mid = 0, mr = 0; i <= n; i++) // 只剩下奇数长度的情况了
        {
            p[i] = 1;
            if(i <= mr)
            {
                p[i] = min(mr - i, p[2 * mid - i]); // 因为 2 * mid - i == ml + mr - i，该式等价于 p[i] = min(mr - i, p[ml + mr - i]);
            }
            while(t[i - p[i]] == t[i + p[i]]) p[i]++;
            if(i + --p[i] > mr) // 最远回文串的左边界，中心，右边界，维护其中的2个就可以了
            {
                ml = i - p[i], mid = i, mr = i + p[i];
            }
            ans = max(ans, p[i]);
        }
        return ans;
    }
    bool check(int l, int r) //判断子串[l,r]是否是回文
    {
        return p[l + r] >= r - l + 1;
    }
} mnc;
```



哈希





哈希算法的应用

COOLMAKER
酷哥创客 人工智能教育

如何辨别哈希题？大致就通过一个特征：

当需要用 O(1) 的时间快速比较两个 O(n) 的数据时，就要考虑使用哈希。

在算法竞赛中哈希大致有以下几种类型（其中最常出现的就是序列哈希）：

- 序列哈希：将序列映射为一个整数，用来快速比较两个序列是否相等。
- 多维序列哈希：用来比较多维序列是否相等，使用容斥原理推理
- 集合哈希：用来快速比较两个集合相等，集合与序列不同点是集合是无序的，也就是说序列要求每个位置相等，而集合只需要对应元素出现次数相等，常先将值映射成大整数，然后将和或者异或的结果**作为哈希函数**解决。
- 树哈希：树上的哈希，一般通过子树转移而来，常用来判断是否同构。
- 图哈希：同树哈希，常用来判断是否同构。
- 数据结构维护哈希值：序列待修改时，需要使用一些合适的数据结构来维护。





序列哈希





3428: 【模板】字符串哈希

题目描述
给定长为 n 的字符串，仅由小写英文字母组成，查找该字符串中不同非空子串的数量。

输入
第一行一个正整数n，代表字符串的长度
第二行一个长度为n的字符串
$1 \le n \le 2*10^3$

输出
输出该字符串中不同非空子串的数量

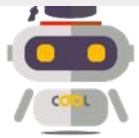
样例输入
```
4
aaab
```

样例输出
```
7
```

提示
字符串aaab 有非空子串a, b, aa, ab, aaa, aab, aaab，一共7 个。

样例二输入
```
16
0100110001010001
```

样例二输出
```
100
```



COOLMAKER 酷哥创客 人工智能教育

思路阶段一：

1. 枚举
2. 通过 内存超限70% 与把子串存进unordered_set中
3. 最后查看集合中元素的个数

```cpp
#include <bits/stdc++.h>
using namespace std;
int n;
string s;
int main()
{
    cin>>n>>s;
    unordered_set<string> st; // 存放截取出来的字符串
    for(int i=0; i<n; i++)//枚举起点
    {
        for(int j=i; j<n; j++)//枚举终点
        {
            string ss=s.substr(i,j-i+1); //从下标i开始截取长度为j-i+1的字符串
            if(!st.count(ss))
            {
                st.insert(ss);
            }
        }
    }
    cout<<st.size();
    return 0;
}
```

3428: 【模板】字符串哈希

题目描述
给定长为 n 的字符串，仅由小写英文字母组成，查找该字符串中不同非空子串的数量。

输入
第一行一个正整数n，代表字符串的长度
第二行一个长度为n的字符串
$1 \le n \le 2*10^3$

输出
输出该字符串中不同非空子串的数量

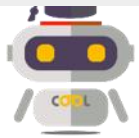
样例输入
```
4
aaab
```

样例输出
```
7
```

提示
字符串aaab 有非空子串a, b, aa, ab, aaa, aab, aaab，一共7 个。

样例二输入
```
16
0100110001010001
```

样例二输出
```
100
```



COOLMAKER 酷哥创客 人工智能教育

思路阶段二：

1. 枚举子串的长度，然后枚举该长度区间的左右端点
2. 通过 时间超限10% 然后把子串
3. 对于每个长度的集合元素的个数进行累加，因为长度不同的元素一定不相同

```cpp
#include <bits/stdc++.h>
using namespace std;
int n;
string s;
int main()
{
    cin>>n>>s;
    int ans=0;
    for(int len=1; len<=n; len++)//枚举长度
    {
        unordered_set<string> st; // 存放截取出来的字符串
        for(int i=0,j=len-1; j<n;i++,j++)//枚举起点、终点
        {
            string ss=s.substr(i,len); //从下标i开始截取长度为len的字符串
            if(!st.count(ss))
            {
                st.insert(ss);
            }
        }
        ans+=st.size();
    }
    cout<<ans;
    return 0;
}
```

序列哈希

COOLMAKER
酷哥创客 人工智能教育

序列哈希把不同的序列映射成不同的整数，最常见的应用就是字符串哈希。

具体到字符串哈希上，我们一般把一个字符串看作一个base进制数，每个字符视为一个数（例如ASCII码），当两个字符串哈希值相等，则两个字符串相等

1.对于一个长度为 n 的字符串 s，

哈希函数定义为：$\text{hash(s)} = \sum_{i=0}^{n-1} s_i base^{n-1-i} \ (\%\ mod)$

例如字符串abc，其哈希值就是$a * base^2 + b * base^1 + c * base^0$

即$97 * base^2 + 98 * base^1 + 99 * base^0$

2.但是哈希有一个很大的弊端：哈希冲突

解决措施：

巧妙设置 base 和 mod 的值，保证base与mod互质，且base和mod都为质数

base的值比序列中点值最大值略大的质数(对于字符串一般取131或13331)

mod的值为一个大质数(一般选取1e9+7或1e9+9或者其他大质数)



序列哈希

A	A	hs[1]=65
AB	$A \times base^1 + B$	hs[2]=hs[1]×base+66
ABC	$A \times base^2 + B \times base^1 + C$	hs[3]=hs[2]×base+67
ABCD	$A \times base^3 + B \times base^2 + C \times base^1 + D$	hs[4]=hs[3]×base+68
ABCDE	$A \times base^4 + B \times base^3 + C \times base^2 + D \times base^1 + E$	hs[5]=hs[4]×base+69

1.求一个字符串的哈希值相当于**求前缀和**
求一个字符串的子串哈希值相当于**求区间和**
两个字符串哈希值相等，相当于两个字符串相等

2.**前缀和** $hs[i] = hs[i-1] \times base + s_{i-1}$，$hs[0] = 0$

3.**区间和** $hash(l,r) = hs[r+1] - hs[l] \times base^{r-l+1}$
例如 $DE = ABCDE - ABC \times base^2 = D \times base^1 + E$
即 $hash(s_3 \sim s_4) = hs[5] - hs[3] \times base^2$

4.计算前缀和，时间复杂度O(n)
查询区间和，时间复杂度O(1)

```
6   const int base=131;
7   const int mod=1e9+7;
8   ll b_pow[2005],hs[2005];
9   void init()
10  {
11      b_pow[0]=1;hs[0]=0;
12      for(int i=1;i<=n;i++)
13      {
14          //计算base的i次方
15          b_pow[i]=b_pow[i-1]*base%mod;
16          //预处理部分hash函数值，hs i代表 0~i-1的字符串hash值
17          hs[i]=(hs[i-1]*base+s[i-1])%mod;
18      }
19  }
20  ll get_hash(int l,int r)
21  {
22      return (hs[r+1]-(hs[l]*b_pow[r-l+1])%mod+mod)%mod;
23  }
```

序列哈希

```
6   const int base=131;
7   const int mod=1e9+7;
8   ll b_pow[2005],hs[2005];
9   void init()
10  {
11      b_pow[0]=1;hs[0]=0;
12      for(int i=1;i<=n;i++)
13      {
14          //计算base的i次方
15          b_pow[i]=b_pow[i-1]*base%mod;
16          //预处理部分hash函数值, hs i代表 0~i-1的字符串hash值
17          hs[i]=(hs[i-1]*base+s[i-1])%mod;
18      }
19  }
20  ll get_hash(int l,int r)
21  {
22      return (hs[r+1]-(hs[l]*b_pow[r-l+1])%mod+mod)%mod;
23  }
```



3428: 【模板】字符串哈希

题目描述
给定长为 n 的字符串，仅由小写英文字母组成，查找该字符串中不同非空子串的数量。

输入
第一行一个正整数n，代表字符串的长度
第二行一个长度为n的字符串
$1 \le n \le 2*10^3$

输出
输出该字符串中不同非空子串的数量

样例输入
```
4
aaab
```

样例输出
```
7
```

提示
字符串aaab 有非空子串a, b, aa, ab, aaa, aab, aaab，一共7 个。

样例二输入
```
16
0100110001010001
```

样例二输出
```
100
```



```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n;
string s;
const int base=131;
const int mod=1e9+7;
ll b_pow[2005],hs[2005];
void init()
{
    b_pow[0]=1;hs[0]=0;
    for(int i=1;i<=n;i++)
    {
        //计算base的i次方
        b_pow[i]=b_pow[i-1]*base%mod;
        //预处理部分hash函数值，hs i代表 0~i-1的字符串hash值
        hs[i]=(hs[i-1]*base+s[i-1])%mod;
    }
}
ll get_hash(int l,int r)
{
    return (hs[r+1]-(hs[l]*b_pow[r-l+1])%mod+mod)%mod;
}
int main()
{
    cin>>n>>s;
    init();
    int ans=0;
    for(int len=1;len<=n;len++)
    {
        unordered_set<ll>st;
        for(int l=0,r=len-1;r<n;l++,r++)
        {
            ll now_hs=get_hash(l,r);
            st.insert(now_hs);
        }
        ans+=st.size();
    }
    cout<<ans;
    return 0;
}
```

3429 问题 C: 【模板】字符串哈希-双哈希

时间限制: 1 Sec 内存限制: 128 MB

提交: 0 解决: 1

[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★☆☆

题目描述

给定长为 n 的字符串，仅由小写英文字母组成，查找该字符串中不同非空子串的数量。

注意：本题与【模板】字符串哈希 完全一样，仅仅只是数据进行了加强卡掉了大部分单哈希的代码，尝试使用多哈希的技巧来通过本题

输入

第一行一个正整数n，代表字符串的长度

第二行一个长度为n的字符串

$1 \le n \le 2*10^3$

输出

输出该字符串中不同非空子串的数量

样例输入

```
4
aaab
```

样例输出

```
7
```

提示

字符串aaab 有非空子串a, b, aa, ab, aaa, aab, aaab，一共7 个。

样例二输入

```
16
0100110001010001
```

样例二输出

```
100
```

COOLMAKER
酷哥创客 人工智能教育

这题和上一题一样，但是我们用刚刚的代码提交，答案错误11%

根据生日悖论随着元素数量的增多，不同元素出现相同值的概率就会以惊人的速度往上增加

如何降低哈希冲突的概率?

双重哈希法

COOLMAKER
酷哥创客 人工智能教育



双重哈希法是解决哈希表冲突的一种策略，它利用两个不同哈希函数来确定元素在哈希表中的位置。当第一个哈希函数产生冲突时，我们不是简单地移动到下一个位置，而是使用第二个哈希函数来决定下一步的移动。



若在单哈希法中发生哈希冲突的概率为$\frac{1}{x}$，使用双哈希后你可以认为此时发生哈希冲突的概率为$\frac{1}{x*x}$，哈希函数是一个压缩映像函数，不可避免的要产生冲突，冲突只能尽可能地减少，而不能完全避免。



3429 问题 C: 【模板】字符串哈希-双哈希

时间限制: 1 Sec 内存限制: 128 MB

提交: 0 解决: 1

[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★☆☆

```cpp
void init()
{
    b_pow[0]=1;hs[0]=0;b_pow2[0]=1;hs2[0]=0;
    for(int i=1;i<=n;i++)
    {
        //计算base的i次方
        b_pow[i]=b_pow[i-1]*base%mod;
        b_pow2[i]=b_pow2[i-1]*base2%mod2;
        //预处理部分hash函数值，hs i代表 0~i-1的字符串hash值
        hs[i]=(hs[i-1]*base+s[i-1])%mod;
        hs2[i]=(hs2[i-1]*base2+s[i-1])%mod2;
    }
}
ll get_hash(int l,int r)
{
    ll h1=(hs[r+1]-(hs[l]*b_pow[r-l+1])%mod+mod)%mod;
    ll h2=(hs2[r+1]-(hs2[l]*b_pow2[r-l+1])%mod2+mod2)%mod2;
    return (h1<<31)+h2;
}
```

样例二输入

```
16
0100110001010001
```

样例二输出

```
100
```

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n;
string s;
const int base=131,base2=13331;
const int mod=1e9+7,mod2=1e9+9;
ll b_pow[2005],hs[2005],b_pow2[2005],hs2[2005];
void init()
{...}
ll get_hash(int l,int r)
{...}
int main()
{
    cin>>n>>s;
    init();
    int ans=0;
    for(int len=1;len<=n;len++)
    {
        unordered_set<ll>st;
        for(int l=0,r=len-1;r<n;l++,r++)
        {
            ll now_hs=get_hash(l,r);
            st.insert(now_hs);
        }
        ans+=st.size();
    }
    cout<<ans;
    return 0;
}
```

3560: 「CSP-S 2023」消消乐

COOLMAKER
酷哥创客 人工智能教育

题目描述

小 L 现在在玩一个低配版本的消消乐，该版本的游戏是一维的，一次也只能消除两个相邻的元素。

现在，他有一个长度为 n 且仅由小写字母构成的字符串。我们称一个字符串是可消除的，当且仅当可以对这个字符串进行若干次操作，使之成为一个空字符串。

其中每次操作可以从字符串中删除两个相邻的相同字符，操作后剩余字符串会拼接在一起。

小 L 想知道，这个字符串的所有非空连续子串中，有多少个是可消除的。

输入

输入的第一行包含一个正整数 n，表示字符串的长度。

输入的第二行包含一个长度为 n 且仅由小写字母构成的的字符串，表示题目中询问的字符串。

输出

输出一行包含一个整数，表示题目询问的答案。

样例输入1 复制

```
8
accabccb
```

样例输出1 复制

```
5
```

【样例 1 解释】

一共有 5 个可消除的连续子串，分别是 `cc`、`acca`、`cc`、`bccb`、`accabccb`。

【数据范围】

对于所有测试数据有：$1 \leq n \leq 2 \times 10^6$，且询问的字符串仅由小写字母构成。

COOLMAKER
酷哥创客 人工智能教育

首先考虑一个串可以被消除时的结构：

- 相邻相同字符 xx 可以被删除
- 若序列 A 和 B 均可以被消除，则 AB 也可以被消除
- 若 A 可以被消除，xAx 也可以被消除

这其实和“合法括号序列”的定义很像，我们可以枚举每一个子串的起点终点，遍历整个子串，维护一个栈来模拟相消的过程来解决。

每当遍历完子串后栈为空，答案加一。

```
for(int i = 1; i <= n; i++)
{
    for(int j = i + 1; j <= n; j++)
    {
        stack<char>pos;
        for(int k = i; k <= j; k++)
        {
            if(!pos.empty() && pos.top() == s[k])
                pos.pop();
            else
                pos.push(s[k]);
        }
        if(pos.empty()) ans++;
    }
}
```

时间复杂度 $O(N^3)$，期望得分 35pts。

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include<bits/stdc++.h>
using namespace std;
int n, ans;
string s;
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> s;
    s = " " + s;
    int ans = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = i + 1; j <= n; j++)
        {
            stack<char>pos;
            for(int k = i; k <= j; k++)
            {
                if(!pos.empty() && pos.top() == s[k])
                    pos.pop();
                else
                    pos.push(s[k]);
            }
            if(pos.empty()) ans++;
        }
    }
    cout << ans << "\n";
    return 0;
}
```

注意到对于起点 L 相同的子串，只要维护过程中栈为空，那么就说明从 L 到当前为止是个“可消除的”的字符串。

aaaaaa

所以我们不需要每次遍历子串中的每一个字符，维护多个栈，只需要遍历一次 L 到 n，维护一个栈来解决起点为 L 的所有子串。

```
for(int i = 1; i <= n; i++)
{
    for(int j = i + 1; j <= n; j++)
    {
        stack<char>pos;
        for(int k = i; k <= j; k++)
        {
            if(!pos.empty() && pos.top() == s[k])
                pos.pop();
            else
                pos.push(s[k]);
        }
        if(pos.empty()) ans++;
    }
}
```



```
for(int i = 1; i <= n; i++)
{
    stack<char>pos;
    for(int j = i; j <= n; j++)
    {
        if(!pos.empty() && pos.top() == s[j])
            pos.pop();
        else
            pos.push(s[j]);
        if(pos.empty()) ans++;
    }
}
```

时间复杂度 $O(N^2)$，期望得分 50pts。

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include<bits/stdc++.h>
using namespace std;
int n, ans;
string s;
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> s;
    s = " " + s;
    int ans = 0;
    for(int i = 1; i <= n; i++)
    {
        stack<char>pos;
        for(int j = i; j <= n; j++)
        {
            if(!pos.empty() && pos.top() == s[j])
                pos.pop();
            else
                pos.push(s[j]);
            if(pos.empty()) ans++;
        }
    }
    cout << ans << "\n";
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育

发现在从 1 到 n 维护栈序列的时候，若对于某个时刻 L 和某个时刻 R，两种时刻的栈序列完全相同，那么说明子串 $[L+1, R]$ 一定是可消除的。

caaa -> ca
caaa -> ca

那么我们可以通过字符串哈希来维护每个时刻的栈序列，可以用 map 或 unordered_map 来维护序列的哈希值的出现次数，前面的每一个相同哈希值位置到当前位置的子串都可以配对消除，若出现了 k 次，那么可以产生 k 个与当前位置匹配的子串，答案加 k。

时间复杂度 $O(N)$，期望得分 100pts。



COOLMAKER
酷哥创客 人工智能教育

```cpp
#include<bits/stdc++.h>
using namespace std;
int n;
string s;
stack<int> pos;
unordered_map<unsigned long long, int> mp;
unsigned long long hs[3000006];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> s;
    s = " " + s;
    long long ans = 0;
    mp[0]++;
    for(int i = 1; i <= n; i++)
    {
        if(!pos.empty() && s[pos.top()] == s[i])
        {
            hs[i] = hs[pos.top() - 1];
            pos.pop();
        }
        else
        {
            hs[i] = hs[i - 1] * 29 + (s[i] - 'a' + 1);
            pos.push(i);
        }
        ans += mp[hs[i]];
        mp[hs[i]]++;
    }
    cout << ans << "\n";
    return 0;
}
```



集合哈希





集合哈希：

用来快速比较两个集合相等，集合与序列不同点是集合是无序的，也就是说序列要求每个位置相等，而集合只需要对应元素出现次数相等，常先将值映射成大整数，然后将和或者异或的结果**作为哈希函数**解决。

积木拼图

COOLMAKER
酷哥创客 人工智能教育

题目描述

小酷宝有两套完全相同的积木套装，分别用序列 A 和 B 表示。每个积木都有一个独特的编号（1 到 N）。

现在小酷宝想知道，能否通过重新排列 A 中某个区间的积木，使其与 B 中某个区间的积木完全匹配。

输入

第一行包含两个整数 N 和 Q，分别表示积木的数量和问题的数量。
第二行包含 N 个整数 $A_1, A_2, \cdots, A_N$，表示第一套积木的编号序列。
第三行包含 N 个整数 $B_1, B_2, \cdots, B_N$，表示第二套积木的编号序列。
接下来 Q 行，每行包含四个整数 l_i, r_i, L_i, R_i，表示第 i 个问题的区间范围。
对于 100% 的数据，$1 \leq N, Q \leq 2 \times 10^5$，$1 \leq A_i, B_i \leq N$，$1 \leq l_i \leq r_i \leq N$，$1 \leq L_i \leq R_i \leq N$。
。

输出

输出 Q 行，每行对应一个问题的答案。如果可以重新排列 A 的 $[l_i, r_i]$ 区间使其与 B 的 $[L_i, R_i]$ 区间完全匹配，输出 `Yes`，否则输出 `No`。

题目描述

小酷宝有两套完全相同的积木套装，分别用序列 A 和 B 表示。每个积木都有一个独特的编号（1 到 N）。

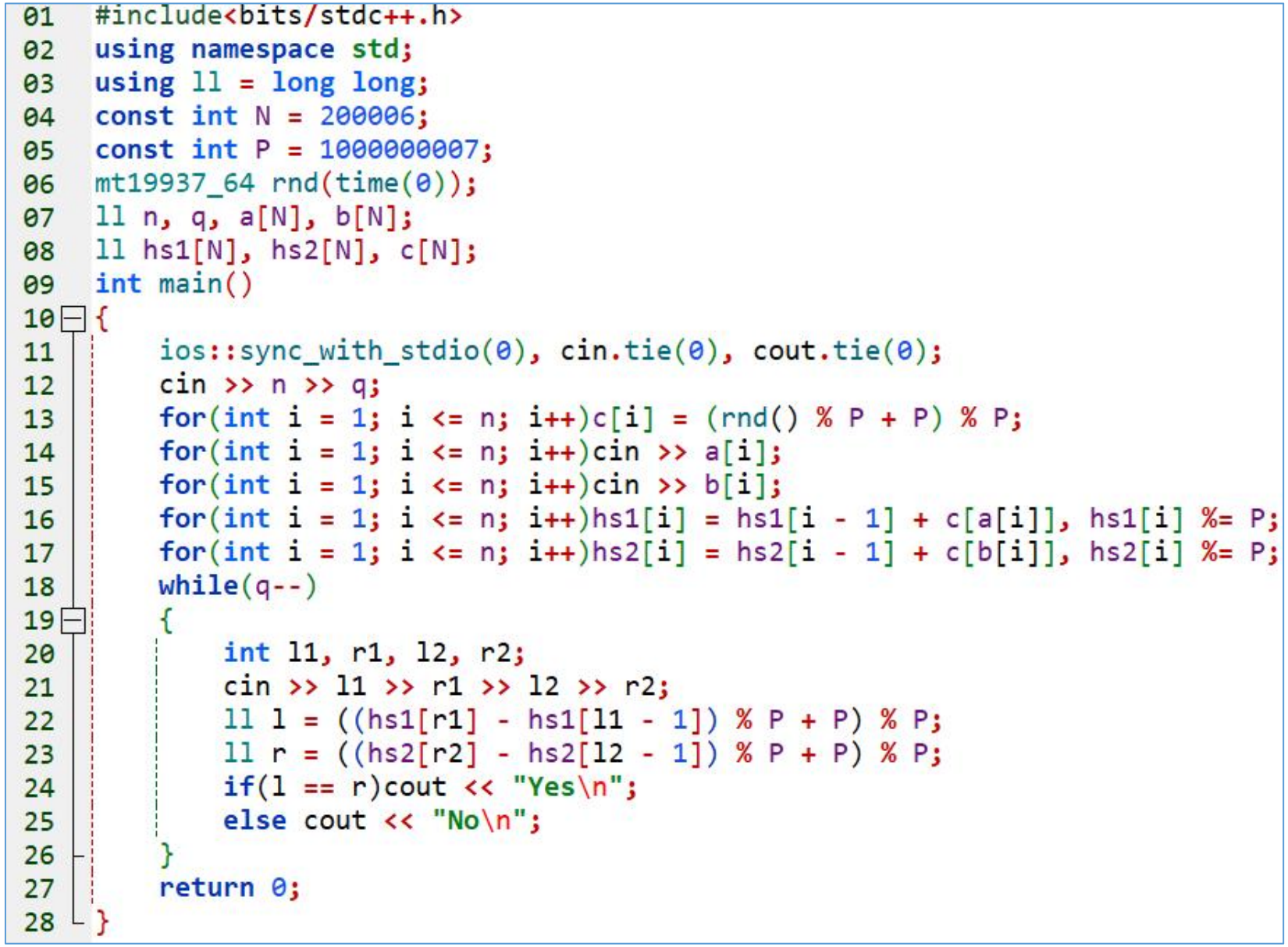
现在小酷宝想知道，能否通过重新排列 A 中某个区间的积木，使其与 B 中某个区间的积木完全匹配。

随机哈希映射：给每个可能的积木编号分配一个随机数（用 mt19937_64 生成，保证随机性），记为 c[x]（x 是积木编号）。这样，一个区间的“元素组成”可以通过区间内所有元素的随机数之和唯一表示（因为每个编号的随机数唯一，和相等当且仅当元素及次数完全相同）。

前缀和预处理：分别对 A 和 B 计算前缀哈希和数组 hs1 和 hs2。其中：
hs1[i] 表示 A[1..i] 所有元素的 c 值之和（模大质数 P 避免溢出）。
hs2[i] 表示 B[1..i] 所有元素的 c 值之和。

查询处理：对于每个查询的区间 $[l_1, r_1]$（A）和 $[L_2, R_2]$（B）：

计算 A 区间的哈希和：$hs1[r_1] - hs1[l_1-1]$（模 P 后调整为非负）。
计算 B 区间的哈希和：$hs2[R_2] - hs2[L_2-1]$。
若两个哈希和相等，说明两个区间元素组成相同（可通过重排匹配），输出 Yes；否则输出 No。



```cpp
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 200006;
const int P = 1000000007;
mt19937_64 rnd(time(0));
ll n, q, a[N], b[N];
ll hs1[N], hs2[N], c[N];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> q;
    for(int i = 1; i <= n; i++)c[i] = (rnd() % P + P) % P;
    for(int i = 1; i <= n; i++)cin >> a[i];
    for(int i = 1; i <= n; i++)cin >> b[i];
    for(int i = 1; i <= n; i++)hs1[i] = hs1[i - 1] + c[a[i]], hs1[i] %= P;
    for(int i = 1; i <= n; i++)hs2[i] = hs2[i - 1] + c[b[i]], hs2[i] %= P;
    while(q--)
    {
        int l1, r1, l2, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        ll l = ((hs1[r1] - hs1[l1 - 1]) % P + P) % P;
        ll r = ((hs2[r2] - hs2[l2 - 1]) % P + P) % P;
        if(l == r)cout << "Yes\n";
        else cout << "No\n";
    }
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育

2627: 星战

COOLMAKER
酷哥创客 人工智能教育

题目描述

在这一轮的星际战争中，我方在宇宙中建立了 n 个据点，以 m 个单向虫洞连接。我们把终点为据点 u 的所有虫洞归为据点 u 的虫洞。

战火纷飞之中这些虫洞很难长久存在，敌人的打击随时可能到来。这些打击中的有效打击可以分为两类：

1.敌人会摧毁某个虫洞，这会使它连接的两个据点无法再通过这个虫洞直接到达，但这样的打击无法摧毁它连接的两个据点。
2.敌人会摧毁某个据点，由于虫洞的主要技术集中在出口处，这会导致该据点的所有还未被摧毁的虫洞被一同摧毁。而从这个据点出发的虫洞则**不会摧毁**。

注意：摧毁只会导致虫洞不可用，而不会消除它的存在。

为了抗击敌人并维护各部队和各据点之间的联系，我方发展出了两种特种部队负责修复虫洞：

1.A 型特种部队则可以将某个特定的虫洞修复。
2.B 型特种部队可以将某据点的所有损坏的虫洞修复。

考虑到敌人打击的特点，我方并未在据点上储备过多的战略物资。因此只要这个据点的某一条虫洞被修复，处于可用状态，那么这个据点也是可用的。

我方掌握了一种苛刻的空间特性，利用这一特性我方战舰可以沿着虫洞瞬移到敌方阵营，实现精确打击。

为了把握发动反攻的最佳时机，指挥部必须关注战场上的所有变化，为了寻找一个能够进行反攻的时刻。总指挥认为：

1.如果从我方的任何据点出发，在选择了合适的路线的前提下，可以进行无限次的虫洞穿梭（可以多次经过同一据点或同一虫洞），那么这个据点就可以**实现反击**。
2.为了使虫洞穿梭的过程连续，尽量减少战舰在据点切换虫洞时的质能损耗，当且仅当**只有一个从该据点出发的虫洞可用**时，这个据点可以**实现连续穿梭**。
3.如果我方所有据点都可以**实现反击**，也都可以**实现连续穿梭**，那么这个时刻就是一个绝佳的**反攻**时刻。

总司令为你下达命令，要求你根据战场上实时反馈的信息，迅速告诉他当前的时刻是否能够进行一次**反攻**。

COOLMAKER
酷哥创客 人工智能教育

输入

输入的第一行包含两个正整数 n, m 。

接下来 m 行每行两个数 u, v ，表示一个从据点 u 出发到据点 v 的虫洞。保证 $u_n ev$ ，保证不会有两条相同的虫洞。初始时所有的虫洞和据点都是完好的。

接下来一行一个正整数 q 表示询问个数。

接下来 q 行每行表示一次询问或操作。首先读入一个正整数 t 表示指令类型：

> *若 $t = 1$ ，接下来两个整数 u, v 表示敌人摧毁了从据点 u 出发到据点 v 的虫洞。保证该虫洞存在且未被摧毁。*

> *若 $t = 2$ ，接下来一个整数 u 表示敌人摧毁了据点 u 。如果该据点的虫洞已全部被摧毁，那么这次袭击不会有任何效果。*

> *若 $t = 3$ ，接下来两个整数 u, v 表示我方修复了从据点 u 出发到据点 v 的虫洞。保证该虫洞存在且被摧毁。*

> *若 $t = 4$ ，接下来一个整数 u 表示我方修复了据点 u 。如果该据点没有被摧毁的虫洞，那么这次修复不会有任何效果。*

在每次指令执行之后，你需要判断能否进行一次反攻。如果能则输出 `YES` 否则输出 `NO` 。

输出

输出一共 q 行。对于每个指令，输出这个指令执行后能否进行反攻。

样例输入1 复制

```
3 6
2 3
2 1
1 2
1 3
3 1
3 2
11
1 3 2
1 2 3
1 1 3
1 1 2
3 1 3
3 3 2
2 3
1 3 1
3 1 3
4 2
1 3 2
```

样例输出1 复制

```
NO
NO
YES
NO
YES
NO
NO
NO
YES
NO
NO
```

【样例解释 _#1】

虫洞状态可以参考下面的图片，图中的边表示存在且未被摧毁的虫洞：

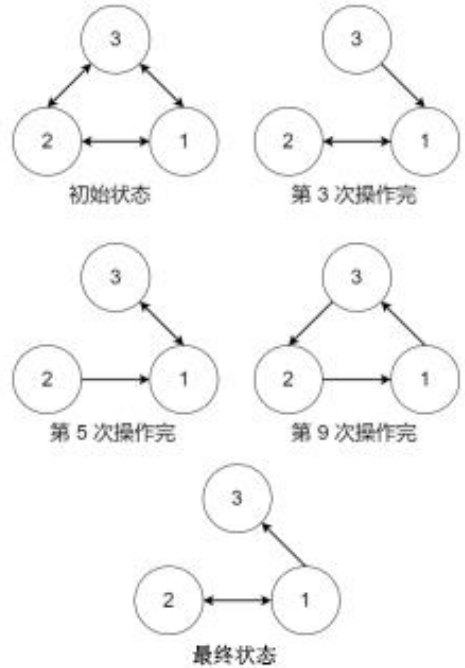


图 1: 样例 1 解释

洛谷

【数据范围】

对于所有数据保证：$1 \leq n \leq 5 \times 10^5$，$1 \leq m \leq 5 \times 10^5$，$1 \leq q \leq 5 \times 10^5$。

n 个点 m 条边的有向图，每条边都有激活和失活两种状态，初始时均为激活状态。四种操作：

1. 失活某条边；
2. 失活以某个点为终点的所有边；
3. 激活某条边；
4. 激活以某个点为终点的所有边。

事实上，所有点出度均为 1 的有向图称作内向基环树森林，环外的点直接或间接地指向环上的点。

然后问：如果只考虑激活的边，是否满足：

- 所有的点出度均为 1；
- 所有的点都满足，从这个点出发，可以走到一个环中。

首先我们发现，如果所有点的出度均为 1，那么所有点都满足从这个点出发能走到一个环里。这是因为所有点的出度都是 1，因此一条路径可以一直走下去，而只要走 n 步就一定会遇到先前走过的一个节点（总共只有 n 个点），此时环就出现了。

因此压根不用判环，只用判断所有点出度是否均为 1。

1.敌人会摧毁某个虫洞，这会使它连接的两个据点无法再通过这个虫洞直接到达，但这样的打击无法摧毁它连接的两个据点。
2.敌人会摧毁某个据点，由于虫洞的主要技术集中在出口处，这会导致该据点的所有还未被摧毁的虫洞被一同摧毁。而从这个据点出发的虫洞则**不会摧毁**。

注意：摧毁只会导致虫洞不可用，而不会消除它的存在。

摧毁据点是摧毁的所有入边，所以直接维护每个点的出度是很难的，考虑将所有的边添加一个出度的编号权值代表这个边是从哪个点出去的，那么维护所有点的出度为 1，相当于维护所有存在的**边权值集合**和 所有**点的编号集合**相同，考虑哈希（和哈希/异或哈希），边的维护可以考虑将这些边权值加到入点上。

设原图上点 u 的入边权值和为 g(u)，当前点 u 入边权值和为 r(u)：

失活 (u,v)：r(v) = r(v)−u；
失活以 v 为终点的所有边：r(v)= 0；
激活 (u,v)：r(v) = r(v)+u；
激活以 v 为终点的所有边：r(v) = g(v)。

这些都可以 O(1) 完成，但是直接和哈希编号很容易被卡（例如存在1条1的出边和2的出边，和存在1条3的出边，权值和是相同的1+2=3，异或哈希同理），因此考虑给每个点的权值从编号改为随机权值。

1.敌人会摧毁某个虫洞，这会使它连接的两个据点无法再通过这个虫洞直接到达，但这样的打击无法摧毁它连接的两个据点。
2.敌人会摧毁某个据点，由于虫洞的主要技术集中在出口处，这会导致该据点的所有还未被摧毁的虫洞被一同摧毁。而从这个据点出发的虫洞则**不会摧毁**。

注意：摧毁只会导致虫洞不可用，而不会消除它的存在。

维护所有存在的**边权值集合** 和 所有**点的编号集合**相同

我们初始给每个点随机一个权值 $w(u)$。重新定义，一个点 v 对应的 $r(v)$，表示直接连向 v 的所有 u 的 $w(u)$ 之和，即：

$$r(v) = \sum_{(u,v)\in E} w(u)$$



COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using ull = unsigned long long;
using namespace std;
const int N = 5e5 + 10;
int n, m, q;
mt19937_64 rnd(114514);
ull stnd, tmp, val[N], sum[N], _sum[N];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>m;
    for(int i = 1; i <= n; ++i)
        stnd += (val[i] = rnd());
    for(int i = 1, u, v; i <= m; ++i)
        cin>>u>>v,sum[v] += val[u], tmp += val[u];
    for(int i = 1; i <= n; ++i)
        _sum[i] = sum[i];
    cin>>q;
    while(q--)
    {
        int op, u, v;
        cin>>op;
        if(op == 1)
            cin>>u>>v, tmp -= val[u], sum[v] -= val[u];
        else if(op == 2)
            cin>>v,tmp -= sum[v], sum[v] = 0;
        else if(op == 3)
            cin>>u>>v, tmp += val[u], sum[v] += val[u];
        else
            cin>>v,tmp += _sum[v] - sum[v], sum[v] = _sum[v];
        cout<<(tmp == stnd? "YES" : "NO")<<"\n";
    }
    return 0;
}
```





KMP算法





字符串基础回顾

COOLMAKER
酷哥创客 人工智能教育

字符串

一个 **字符串** S 是将 n 个字符顺次排列形成的序列，n 称为 S 的长度，表示为 $|S|$。

如果字符串下标从 1 开始计算，S 的第 i 个字符表示为 $S[i]$；

如果字符串下标从 0 开始计算，S 的第 i 个字符表示为 $S[i-1]$。

子串

字符串 S 的 **子串** $S[i..j]$，$i \le j$，表示 S 串中从 i 到 j 这一段，也就是顺次排列 $S[i], S[i+1], \ldots, S[j]$ 形成的字符串。

有时也会用 $S[i..j]$，$i > j$ 来表示空串。

子序列

字符串 S 的 **子序列** 是从 S 中将若干元素提取出来并不改变相对位置形成的序列，即 $S[p_1], S[p_2], \ldots, S[p_k]$，$1 \le p_1 < p_2 < \cdots < p_k \le |S|$。



字符串基础回顾

COOLMAKER
酷哥创客 人工智能教育

前缀

前缀 是指从串首开始到某个位置 i 结束的一个特殊子串。字符串 S 的以 i 结尾的前缀表示为 $Prefix(S,i)$，也就是 $Prefix(S,i) = S[0..i]$。

真前缀 指除了 S 本身的 S 的前缀。

举例来说，字符串 `abcabcd` 的所有前缀为 `{a, ab, abc, abca, abcab, abcabc, abcabcd}`，而它的真前缀为 `{a, ab, abc, abca, abcab, abcabc}`。

后缀

后缀 是指从某个位置 i 开始到整个串末尾结束的一个特殊子串。字符串 S 的从 i 开头的后缀表示为 $Suffix(S,i)$，也就是 $Suffix(S,i) = S[i..|S|-1]$。

真后缀 指除了 S 本身的 S 的后缀。

举例来说，字符串 `abcabcd` 的所有后缀为 `{d, cd, bcd, abcd, cabcd, bcabcd, abcabcd}`，而它的真后缀为 `{d, cd, bcd, abcd, cabcd, bcabcd}`。



KMP算法

COOLMAKER
酷哥创客 人工智能教育

KMP算法（全称Knuth-Morris-Pratt字符串查找算法，由三位发明者的姓氏命名）是通过使用前缀函数可以在文本串s中快速查找模式串p的一种算法。时间复杂度是O(n)。

举例来说，假如文本串s="abababcabaa", 模式串p="ababcabaa"

我们想要知道文本串s中有多个位置能够和模式串p匹配，或者说模式串p在文本串s中出现了多少次?



KMP算法

COOLMAKER
酷哥创客 人工智能教育

暴力做法

要想知道KMP算法是如何减少字符串查找的时间复杂度的，我们不如来看暴力匹配方法是如何浪费时间的。

- 所谓暴力匹配，就是逐字符逐字符地进行匹配（比较s[i]和p[j]）
- 如果当前字符匹配成功（s[i]==p[j]），就匹配下一个字符（++i, ++j）
- 如果失配，i回溯，j置为0（i=i-j+1, j=0）。
- 如此往复，直到 s 或 p 中所有字符比较完毕。

```
// 暴力匹配
int i = 0, j = 0;
while (i < s.length())
{
    if (s[i] == p[j])
        ++i, ++j;
    else
        i = i - j + 1, j = 0;
    if (j == p.length())  // 匹配成功
    {
        // 对s[i - j .. i - 1]进行一些操作
        cout << i - j << endl;
        i = i - j + 1;
        j = 0;
    }
}
```



KMP算法

COOLMAKER
酷哥创客 人工智能教育

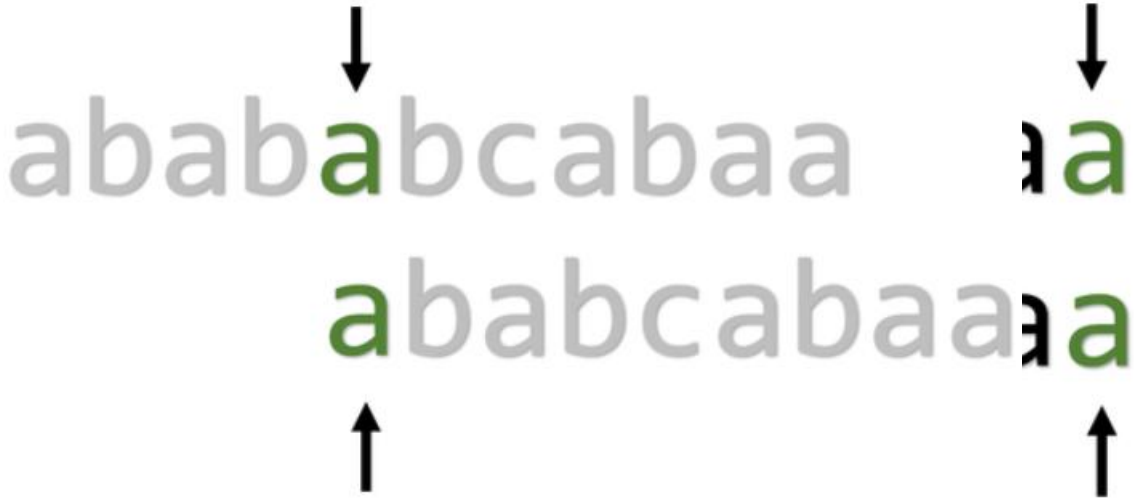
逐字符逐字符地进行匹配（比较s[i]和p[j]）

如果当前字符匹配成功（s[i]==p[j]），就匹配下一个字符（++i, ++j）

如果失配，i回溯，j置为0（i=i-j+1, j=0）。

```
// 暴力匹配
int i = 0, j = 0;
while (i < s.length())
{
    if (s[i] == p[j])
        ++i, ++j;
    else
        i = i - j + 1, j = 0;
    if (j == p.length())  // 匹配成功
    {
        // 对s[i - j .. i - 1]进行一些操作
        cout << i - j << endl;
        i = i - j + 1;
        j = 0;
    }
}
```

设两个字符串的长度分别为 n 和 m，则暴力匹配的最坏时间复杂度是 $O(nm)$。究其原因，在于i进行回溯浪费了时间。能不能让i不走回头路呢？然而，如果i不回溯，同时又把 `j` 置为0，很可能会出现缺漏，如下图。



这样配下去会漏掉一个匹配

就这样一直匹配到结尾。

KMP算法

COOLMAKER
酷哥创客 人工智能教育

文本串s="abababcabaa", 模式串p="ababcabaa"

于是为了让 j 被赋为一个合适的值，使得让s[i]不回退，p[j] 回退的同时不会漏掉匹配，引入了前缀函数，并得到了一张表，我们把这张表称作PMT（Partial Match Table，部分匹配表）。

我们通常用π表示前缀函数， π[i] 就是模式串 p[0 ~ i] 最长的相等的真前缀与真后缀的长度。简单地说，π[i]就是从p[0]往后数、同时从p[i]往前数相同的位数，在保证前后缀相同的情况下，最多能数多少位，但是要小于p的长度。

位置	0	1	2	3	4	5	6	7	8
模式串p	a	b	a	b	c	a	b	a	a
前缀函数π	0	0	1	2	0	1	2	3	1

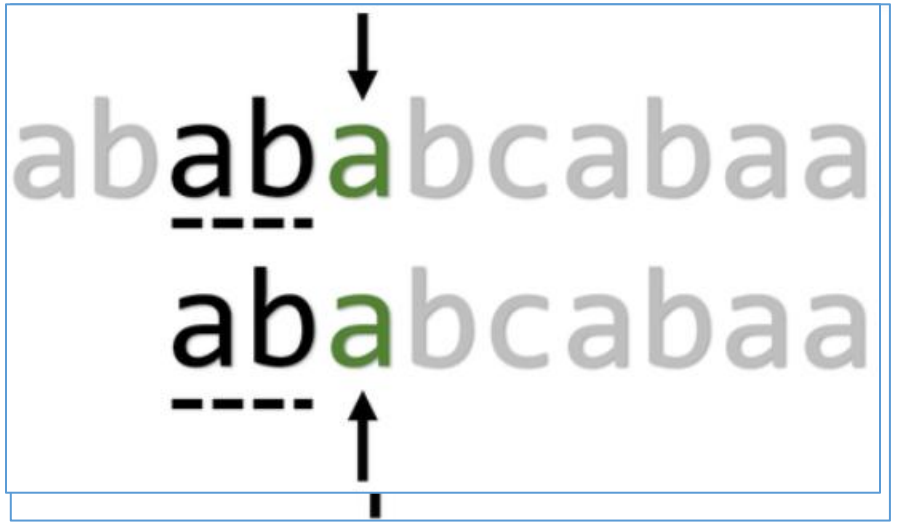


KMP算法

COOLMAKER
酷哥创客 人工智能教育

位置	0	1	2	3	4	5	6	7	8
模式串p	a	b	a	b	c	a	b	a	a
前缀函数π	0	0	1	2	0	1	2	3	1

文本串s="abababcabaa", 模式串p="ababcabaa"
得到了前缀函数有什么用呢？让我们先回到暴力匹配算法第一次失配时的情形



这个时候，文本串s中的’a’与模式串p中的’c’没有配上，我们计划保存i指针（上面的指针）不变，而把j指针左移。我们注意到，”abab”已经匹配成功了，它拥有一个前缀”ab”，以及一个后缀”ab”（虚线部分），所以我们可以吧这个”ab”利用起来

变成这个样子，也就是让 j=π[j-1]
如果s[i]仍然不等于p[j]，那么继续让j=π[j-1]，最后如果都不匹配，那么最终j=0

KMP算法

COOLMAKER
酷哥创客 人工智能教育

求文本匹配

```
for (int i = 0, j = 0; i < s.length(); ++i)
{
    while (j && s[i] != p[j]) j = pmt[j - 1]; // 不断前移j指针，直到成功匹配或移到头为止
    if (s[i] == p[j]) j++;  // 当前位匹配成功，j指针右移
    if (j == p.length())//找到一个完全匹配的位置[ i-p.length+1, i]
    {
        cout<<i-p.length()+1<<" "<<i<<endl;
        j = pmt[j - 1];
    }
}
```



KMP算法

COOLMAKER
酷哥创客 人工智能教育

前缀函数是KMP算法的核心部分，KMP算法通过前缀函数来实现高效的字符串匹配。

KMP算法在字符串匹配过程中，利用前缀函数来跳过一些明显不会产生匹配的比较，从而提高字符串匹配的效率。当模式串与目标串不匹配时，KMP算法会根据前缀函数来决定模式串应该从哪个位置开始重新比较，而不是从头开始比较。



KMP算法

COOLMAKER 酷哥创客 人工智能教育

前缀函数

定义

给定一个长度为 n 的字符串 s，其 **前缀函数** 被定义为一个长度为 n 的数组 π。 其中 $\pi[i]$ 的定义是：

1. 如果子串 $s[0 \dots i]$ 有一对相等的真前缀与真后缀：$s[0 \dots k-1]$ 和 $s[i-(k-1) \dots i]$，那么 $\pi[i]$ 就是这个相等的真前缀（或者真后缀，因为它们相等））的长度，也就是 $\pi[i] = k$；
2. 如果不止有一对相等的，那么 $\pi[i]$ 就是其中最长的那一对的长度；
3. 如果没有相等的，那么 $\pi[i] = 0$。

简单来说 $\pi[i]$ 就是，子串 $s[0 \dots i]$ 最长的相等的真前缀与真后缀的长度。

用数学语言描述如下：

$$\pi[i] = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

特别地，规定 $\pi[0] = 0$。

过程

举例来说，对于字符串 `abcabcd`，

$\pi[0] = 0$，因为 `a` 没有真前缀和真后缀，根据规定为 0

$\pi[1] = 0$，因为 `ab` 无相等的真前缀和真后缀

$\pi[2] = 0$，因为 `abc` 无相等的真前缀和真后缀

$\pi[3] = 1$，因为 `abca` 只有一对相等的真前缀和真后缀：`a`，长度为 1

$\pi[4] = 2$，因为 `abcab` 相等的真前缀和真后缀只有 `ab`，长度为 2

$\pi[5] = 3$，因为 `abcabc` 相等的真前缀和真后缀只有 `abc`，长度为 3

$\pi[6] = 0$，因为 `abcabcd` 无相等的真前缀和真后缀

同理可以计算字符串 `aabaaab` 的前缀函数为 $[0, 1, 0, 1, 2, 2, 3]$。



重要的观察是 **相邻的前缀函数值至多增加 1**。

KMP算法

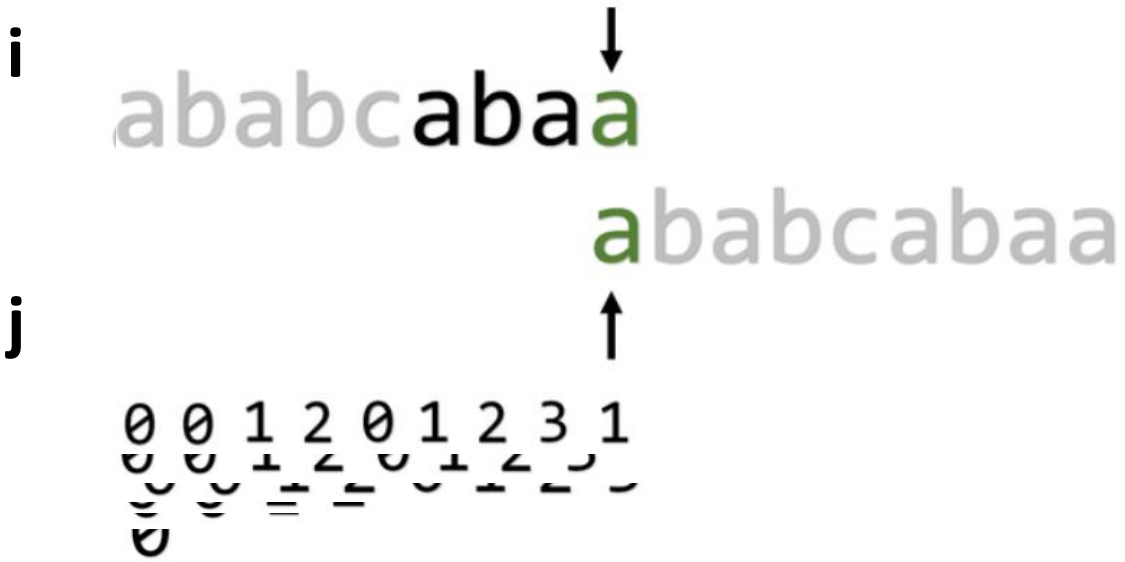
COOLMAKER
酷哥创客 人工智能教育

求前缀函数

和求文本匹配的方式一样
方法：在错开一位后，让模式串p自己匹配自己（这相当于是用前缀去匹配后缀）。我们知道pmt[0]=0，而之后的每一位则可以通过在匹配过程中记录j值得到。

再次匹配，匹配成功，j指针右移一位，pmt[i]=j=1。自此，我们通过一趟自我匹配

```
pmt[0] = 0;
for (int i = 1, j = 0; i < p.length(); ++i)
{
    while (j && p[i] != p[j]) j = pmt[j - 1];
    if (p[i] == p[j]) j++;
    pmt[i] = j;
}
```



KMP算法

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
const int maxn=1e6+5;
int pmt[maxn];
void get_pmt(string p)
{
    pmt[0] = 0;
    for (int i = 1, j = 0; i < p.length(); ++i)
    {
        while (j && p[i] != p[j]) j = pmt[j - 1];
        if (p[i] == p[j]) j++;
        pmt[i] = j;
    }
}
void kmp(string s,string p)
{
    int cnt=0;
    for (int i = 0, j = 0; i < s.length(); ++i)
    {
        while (j && s[i] != p[j]) j = pmt[j - 1]; // 不断前移j指针，直到成功匹配或移到头为止
        if (s[i] == p[j]) j++;  // 当前位匹配成功，j指针右移
        if (j == p.length())//找到一个完全匹配的位置[ i-p.length+1, i]
        {
            //          cout<<i-p.length()+1<<" "<<i<<endl;
            cnt++;
            j = pmt[j - 1];
        }
    }
    cout<<cnt;
}
int main()
{
    string s,p;
    cin>>s>>p;
    get_pmt(p);//得到模式串p的前缀函数
    kmp(s,p);//查找p在s中的出现位置
    return 0;
}
```



KMP算法

COOLMAKER
酷哥创客 人工智能教育

字符串的周期

对字符串 s 和 $0 < p \le |s|$，若 $s[i] = s[i+p]$ 对所有 $i \in [0, |s| - p - 1]$ 成立，则称 p 是 s 的周期。

对字符串 s 和 $0 \le r < |s|$，若 s 长度为 r 的前缀和长度为 r 的后缀相等，就称 s 长度为 r 的前缀是 s 的 border。

由 s 有长度为 r 的 border 可以推导出 $|s| - r$ 是 s 的周期。

根据前缀函数的定义，可以得到 s 所有的 border 长度，即 $\pi[n-1], \pi[\pi[n-1]-1], \ldots$。

所以根据前缀函数可以在 $O(n)$ 的时间内计算出 s 所有的周期。其中，由于 $\pi[n-1]$ 是 s 最长 border 的长度，所以 $n - \pi[n-1]$ 是 s 的最小周期。

|s| - 最长border = 最小周期，最长border=π[n-1]，最小周期=n-π[n-1]
|s| - 最短border = 最大周期

KMP算法

COOLMAKER
酷哥创客 人工智能教育

2572: 重复子串

题目描述
给定若干个的字符串，
ababab 最多由 个 ab

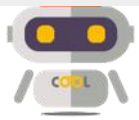
输入
输入若干行，每行有一
号，此时输入结束。

输出
输出每个字符串最多是
对于100%的数据，满

样例输入
```
abcd
aaaa
ababab
.
```

样例输出
```
1
4
3
```

```cpp
#include <bits/stdc++.h>
using namespace std;
const int maxn=1e6+5;
int pmt[maxn];
void get_pmt(string p)
{
    pmt[0] = 0;
    for (int i = 1, j = 0; i < p.length(); ++i)
    {
        while (j && p[i] != p[j]) j = pmt[j - 1];
        if (p[i] == p[j]) j++;
        pmt[i] = j;
    }
    int n=p.length();
    if(n%(n-pmt[n-1])==0)cout<<n/(n-pmt[n-1])<<"\n";
    else cout<<1<<"\n";
}
int main()
{
    string p;
    while(cin>>p)
    {
        if(p==".")break;
        get_pmt(p);//得到模式串p的前缀函数
    }
    return 0;
}
```



最多是由多少个相同的子字符串重复连接而成，等价于求字符串的最小周期，用周期串来进行重复连接，看是否能构成原串

如果 |s|%最小周期==0，代表可以使用这个最小周期串可以拼接成原字符串。

例如 S=ababab，该字符串的周期串有ab，abab

最小周期串为ab，长度为2，也就是最小周期为2

|s|=6，6%2==0，所以可以用ab来拼接，需要6/2=3个ab串来拼接，输出3

例如S=ababa，该字符串的周期串有ab，abab

最小周期串为ab，长度为2，也就是最小周期为2

|s|=5，5%2==1，所以不可以用周期串来拼接，所以只能用ababa原字符串，输出1



Trie树

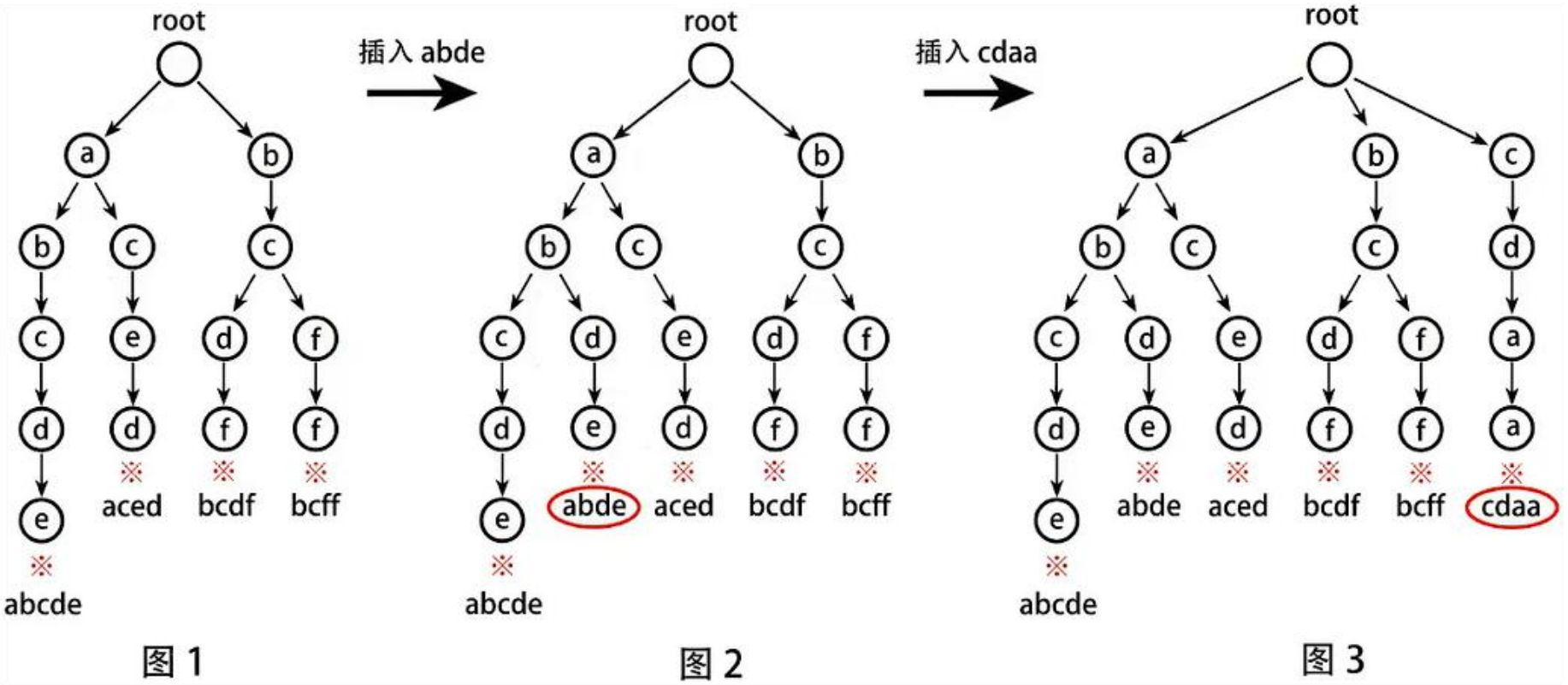




什么是Trie树

COOLMAKER
酷哥创客 人工智能教育

Trie树，又称字典树、单词查找树、前缀树（Prefix Tree）或 键树；是一种多叉树结构，每个节点保存一个字符，一条路径表示一个字符串。

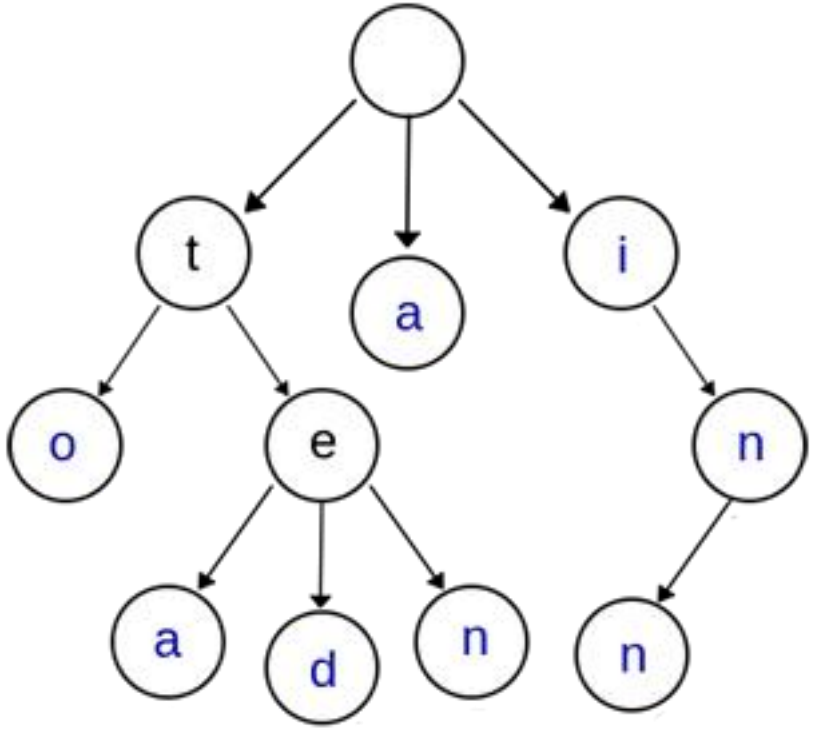


Trie树基本性质

COOLMAKER
酷哥创客 人工智能教育

右图是一棵Trie树，表示了关键字集合{“a”，“to”，“tea”，“ted”，“ten”，“i”，“in”，“inn”}。从上图可以归纳出Trie树的基本性质：

1. 根节点不包含字符，除根节点外的每一个子节点都包含一个字符。
2. 从根节点到一个节点，路径上经过的字符连接起来，为该节点对应的字符串。
3. 每个节点的所有子节点包含的字符互不相同。
4. 在实现的时候，会在节点结构中设置一个标志，用来标记该结点处是否构成一个单词（关键字）。如右图蓝色节点代表集合中的单词。
5. 有公共前缀的字符串，他们的公共前缀会共用节点。例如”tea”、”ted”共用节点”t”、”e”





字典树的功能

COOLMAKER
酷哥创客 人工智能教育

根据字典树的概念，我们可以发现：字典树的本质是把很多字符串拆成单个字符的形式，以树的方式存储起来。所以我们说字典树维护的是”字典“。那么根据这个最基本的性质，我们可以由此延伸出字典树的很多妙用。简单总结起来大体如下：

1、维护字符串集合（即字典）。

2、向字符串集合中插入字符串（即建树）。

3、查询字符串集合中是否有某个字符串（即查询，字符串结尾节点值+1）。

4、统计字符串在集合中出现的个数（即统计，路径值+1）。

5、将字符串集合按字典序排序（即字典序排序，dfs遍历）。

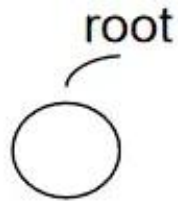


Trie树的构建

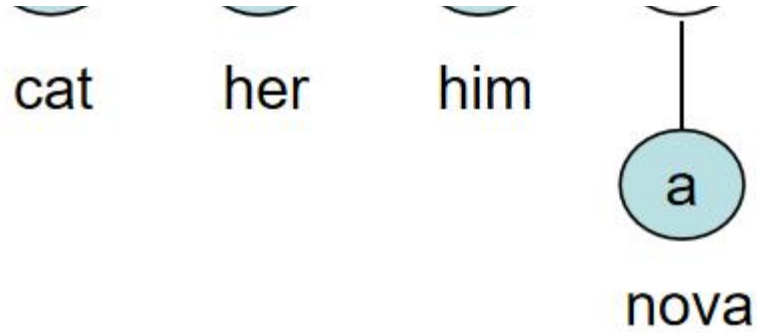
COOLMAKER
酷哥创客 人工智能教育

Trie 树的构建过程，就是不断将字符串插入树中。
以插入字符串 him 、 her 、 cat 、 no 、 nova 为例，过程如下：
1. 插入 him :

- 根节点不存在子节点 h，因此创建子节点 h。
- 在节点 h 的基础上插入第二个字符 i。
- 节点 h 不存在子节点 i，创建子节点 i。
- 在节点 i 的基础上插入第三个字符 m。
- 节点 i 不存在子节点 m，创建子节点 m。并将该节点标记为字符串结束标志，完成 him 字符串插入。



him
her
cat
no
nova



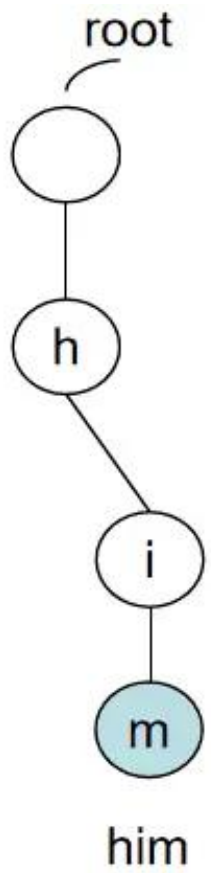
Trie树的构建

COOLMAKER
酷哥创客 人工智能教育

Trie 树的构建过程，就是不断将字符串插入树中。
以插入字符串 him 、 her 、 cat 、 no 、 nova 为例，过程如下：
2. 插入 her :

- 根节点存在子节点 h。不用重新创建子节点 h。
- 在节点 h 的基础上插入第二个字符 e。
- 节点 h 不存在子节点 e，创建子节点 e。
- 在节点 e 的基础上插入第三个字符 r。
- 节点 e 不存在子节点 r，创建子节点 r。并将该节点标记为字符串结束标志，完成 her 字符串插入。

him
her
cat
no
nova



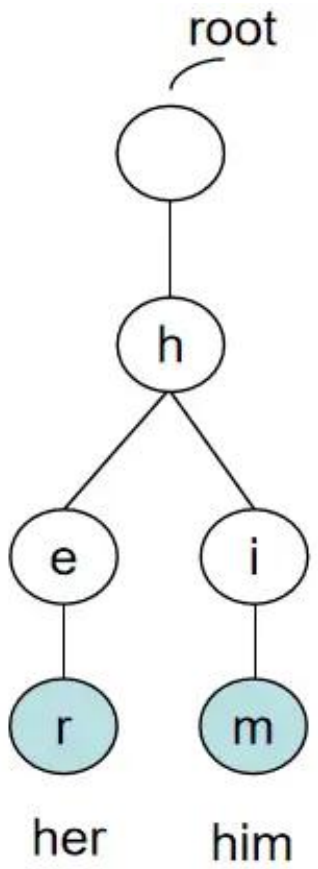
Trie树的构建

COOLMAKER
酷哥创客 人工智能教育

Trie 树的构建过程，就是不断将字符串插入树中。
以插入字符串 him 、 her 、 cat 、 no 、 nova 为例，过程如下：
3. 插入 cat：

- 根节点不存在子节点 c，因此创建子节点 c。
- 在节点 c 的基础上插入第二个字符 a。
- 节点 c 不存在子节点 a，创建子节点 a。
- 在节点 a 的基础上插入第三个字符 t。
- 节点 a 不存在子节点 t，创建子节点 t。并将该节点标记为字符串结束标志，完成 cat 字符串插入。

him
her
cat
no
nova

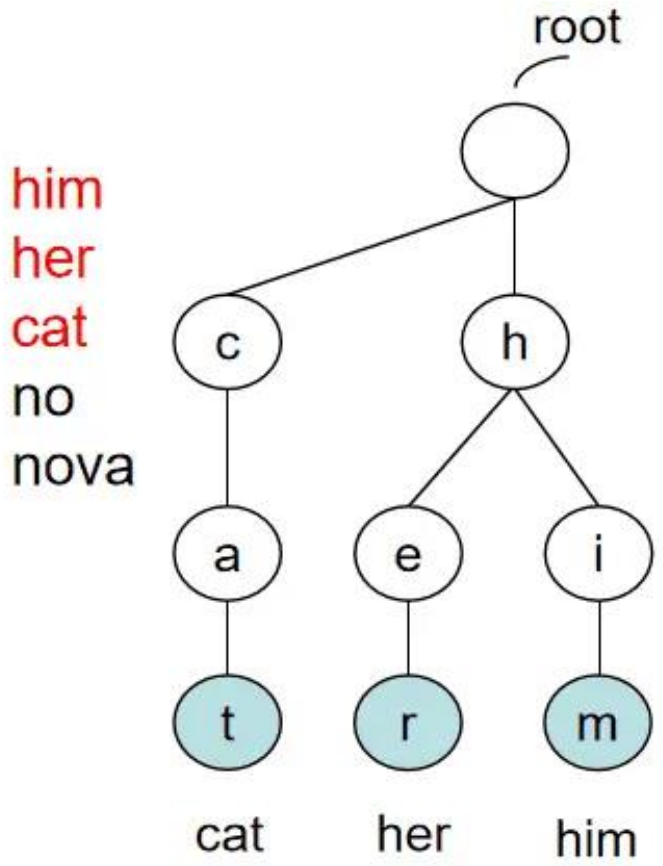


Trie树的构建

COOLMAKER
酷哥创客 人工智能教育

Trie 树的构建过程，就是不断将字符串插入树中。
以插入字符串 him 、 her 、 cat 、 no 、 nova 为例，过程如下：
4. 插入 no:

- 根节点不存在子节点 n，因此创建子节点 n。
- 在节点 n 的基础上插入第二个字符 o。
- 节点 n 不存在子节点 o，创建子节点 o。并将该节点标记为字符串结束标志，完成 no 字符串插入。

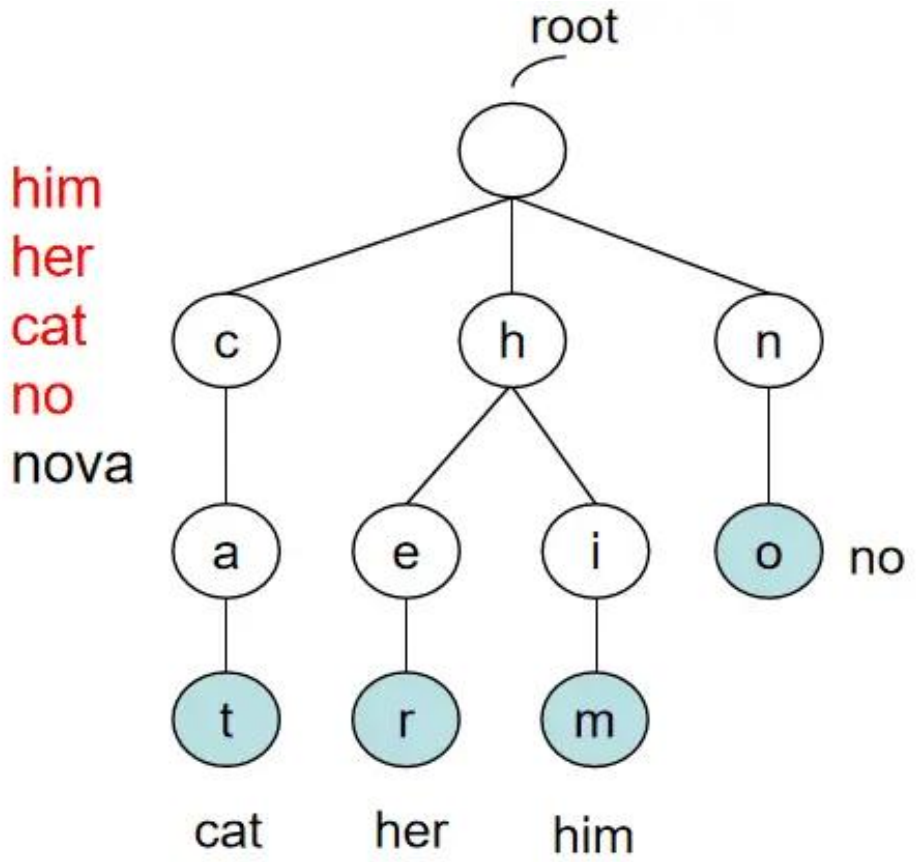


Trie树的构建

COOLMAKER
酷哥创客 人工智能教育

Trie 树的构建过程，就是不断将字符串插入树中。
以插入字符串 him 、 her 、 cat 、 no 、 nova 为例，过程如下：
5. 插入 nova:

- 根节点存在子节点 n，不用重新创建子节点 n。
- 在节点 n 的基础上插入第二个字符 o。
- 节点 n 存在子节点 o，不用重新创建子节点 o。 - 在节点 o 的基础上插入第三个字符 v。
- 节点 o 不存在子节点 v，创建子节点 v。
- 在节点 v 的基础上插入第四个字符 a。
- 节点 v 不存在子节点 a，创建子节点 a。并将该节点标记为字符串结束标志，完成 nova 字符串插入。



Trie树的实现

COOLMAKER
酷哥创客 人工智能教育

```
int nex[1000005][26], tot=0;
int cnt[1000005];  // 该结点结尾的字符串是否存在,记录出现次数
void insert(string s, int len)    // 插入字符串
{
    int p = 0;
    for(int i = 0; i < len; i++)
    {
        int c = s[i] - 'a';
        if(!nex[p][c]) nex[p][c] = ++tot;   // 如果没有，就添加结点
        p = nex[p][c];
    }
    cnt[p] += 1;  // 标记 该结点结尾的字符串存在
}
```



Trie树的查询

COOLMAKER
酷哥创客 人工智能教育

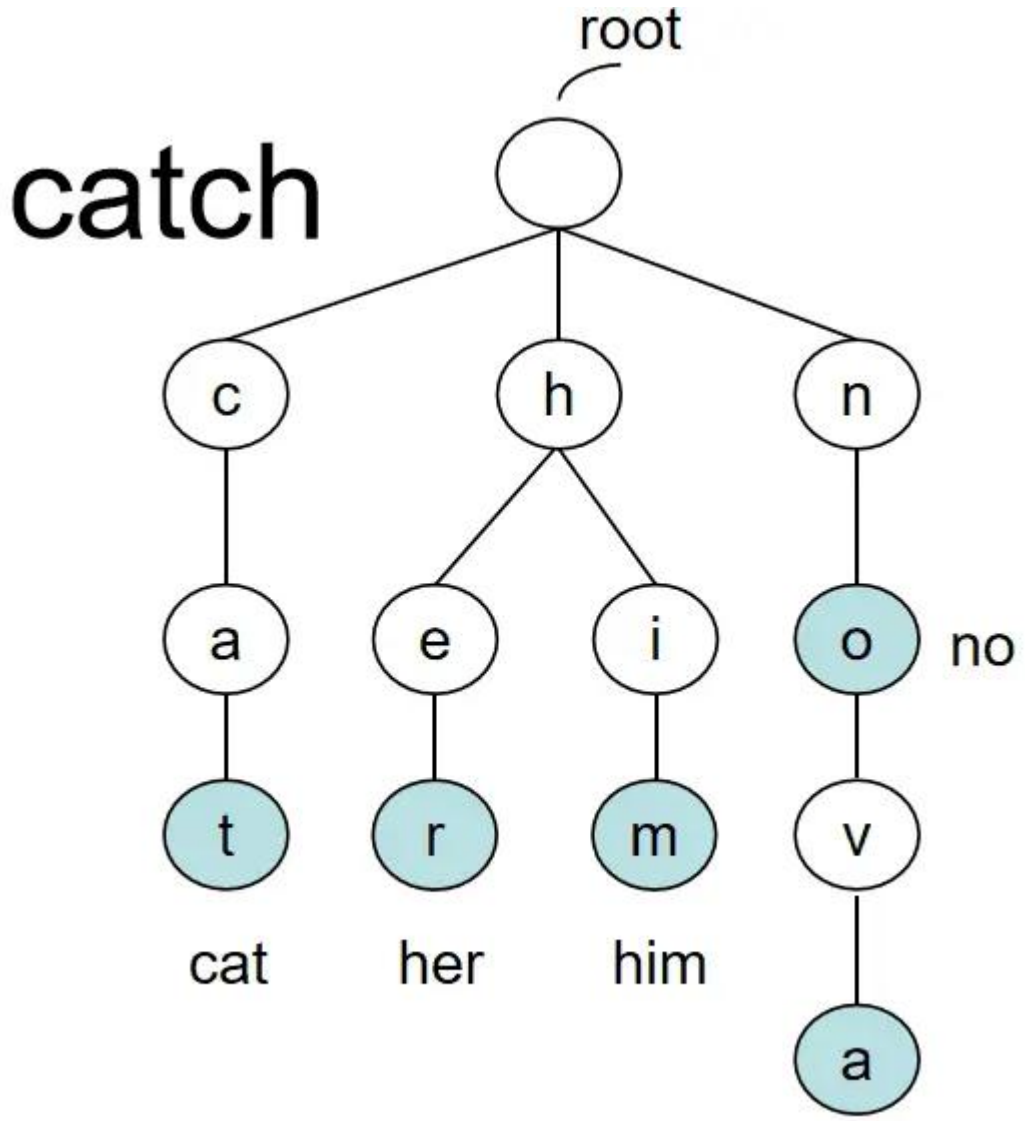
Trie 树最基本的作用是在树上查找字符串。

例如有 5 个字符串： him 、 her 、 cat 、 no 、 nova 。现在要查找 catch 是否存在。

如果使用暴力的方法，需要用 catch 与这 5 个字符串分别进行匹配，效率较低。

如果将这 5 个字符串存储成 Trie 的结构，只需要顺着路径依次比较。

比较完 cat 之后，没有节点与 c 匹配，所以字符串集合中不存在 catch。



Trie树的实现

COOLMAKER
酷哥创客 人工智能教育

查询是否存在字符串
1.从根开始查，扫描字符串。
2.有字母s[i]，则走下来，能走到词尾，则返回插入次数cnt[p]。
3.无字母s[i]，则返回0。

```
bool query(string s, int len)     // 查找字符串
{
    int p = 0;
    for(int i = 0; i < len; i++)
    {
        int c = s[i] - 'a';
        if(!nex[p][c]) return 0;    // 如果没有，代表字典树中没有这个单词
        p = nex[p][c];
    }
    return cnt[p];  // 返回该节点的标记信息
}
```



Trie树的实现

COOLMAKER 酷哥创客 人工智能教育

离线建树

```
3   struct trie
4   {
5       int nex[1000005][26], tot=0;
6       int cnt[1000005];  // 该结点结尾的字符串是否存在,记录出现次数
7       void insert(string s, int len)    // 插入字符串
8       {
9           int p = 0;
10          for(int i = 0; i < len; i++)
11          {
12              int c = s[i] - 'a';
13              if(!nex[p][c]) nex[p][c] = ++tot;
14              p = nex[p][c];
15          }
16          cnt[p] += 1;  // 标记 该结点结尾的字符
17      }
18      bool query(string s, int len)    // 查找字
19      {
20          int p = 0;
21          for(int i = 0; i < len; i++)
22          {
23              int c = s[i] - 'a';
24              if(!nex[p][c]) return 0;   // 如果没有，代表字典树中没有这个单词
25              p = nex[p][c];
26          }
27          return cnt[p];  // 返回该节点的标记信息
28      }
29  } tree;
```

离线建树优点：

- 运行快
- 内存消耗小

在线建树优点：

- 不需要根据题目来估算数组大小

在线建树

```
3   struct trie
4   {
5       struct trie_node
6       {
7           int cnt = 0; //标志，0：不是字符串结尾，>0：记录出现次数
8           trie_node* child[26] = {nullptr}; //指向子节点指针
9       };
10      trie_node* root=new trie_node;
11      void insert(string s, int len)//插入字符串
12      {
13          trie_node* p = root;
14          for(int i = 0; i < len; i++)
15          {
16              int n = s[i] - 'a';
17              if(!p->child[n]) //没有对应子节点，创建
18              {
19                  trie_node* q =new trie_node;
20                  p->child[n] = q;
21              }
22              p = p->child[n];
23          }
24          p->cnt += 1;//字符串结尾标志位置为1
25      }
26      bool query(string s, int len)//检索字符串
27      {
28          trie_node* p = root;
29          for(int i = 0; i < len; i++)
30          {
31              int n = s[i] - 'a';
32              if(!p->child[n])return 0;
33              p = p->child[n];
34          }
35          return p->cnt;
36      }
37  } tree;
```

2580: 前缀统计

时间限制: 1 Sec 内存限制: 128 MB

提交: 0 解决: 1

[提交] [状态] [讨论版] [命题人: hjw] [Edit] [TestData]

难度

★★★☆☆

题目描述

给定N个字符串S_1,S_2.....S_N，接下来进行M次询问，每次询问给定一个字符串T，求S_1~S_N中有多少个字符串是T的前缀。

输入

第一行输入两个整数N，M。
接下来N行每行输入一个字符Si。
接下来M行每行一个字符串T用以询问。

输出

对于每个询问，输出一个整数表示答案。
每个答案占一行。

样例输入

```
3 2
ab
bc
abc
abc
efg
```

样例输出

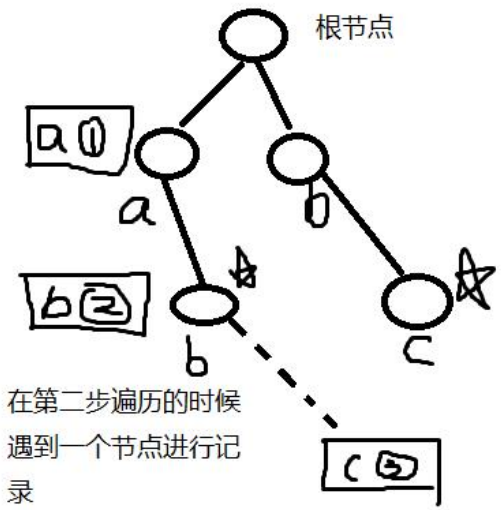
```
2
0
```

提示

对于 100% 的数据，$1 \le N \le 10^6$，$1 \le M \le 100$，$1 \le \sum_{i=1}^{N} |S_i| \le 10^6$，$S$ 仅包含小写字母。

然后我们每次查询由查询的字符串按照建立的Tire进行遍历，找到包含字串的个数：

以abc字符串为例：



在第二步遍历的时候遇到一个节点进行记录

第三步遍历的时候不存在，直接返回遍历。

https://blog.csdn.net/qq_42635159

Trie树的实现

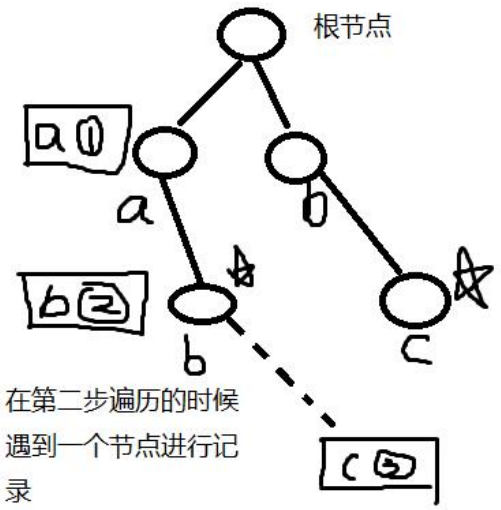
离线建树

在线建树

首先我们根据被查询的对象建立Tire,并且每一个字符串的结尾都做一个标记（也就是用数字记录个数）：

我们每次查询由查询的字符串按照建立的Tire进行遍历，找到包含字串的个数：

以abc字符串为例：



在第二步遍历的时候遇到一个节点进行记录

第三步遍历的时候不存在，直接返回遍历。



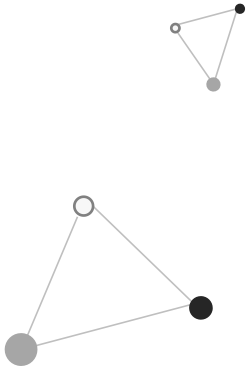
https://blog.csdn.net/qq_42635159

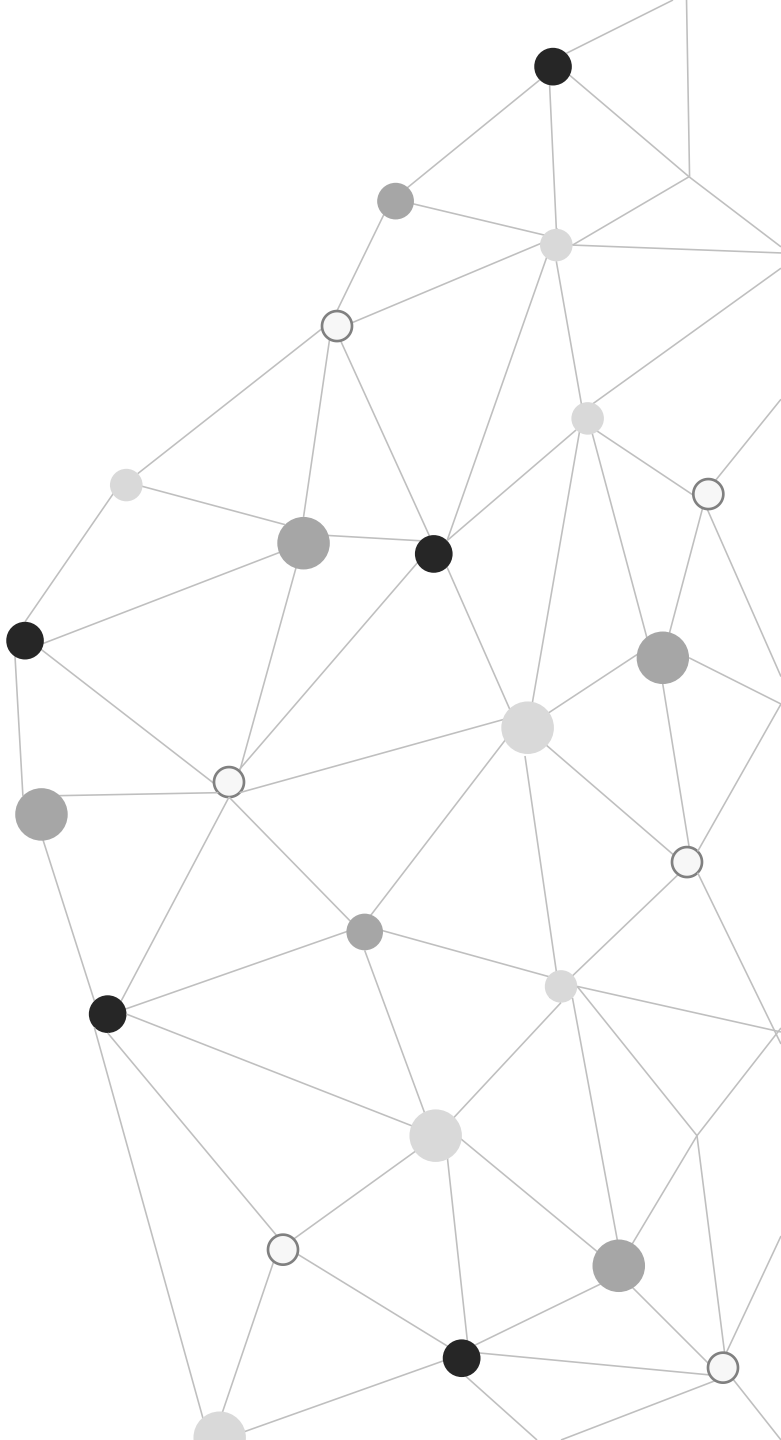
```
#include <bits/stdc++.h>
using namespace st
struct trie
{
    int nex[100000
    int cnt[100000
    void insert(st
    {
        int p = 0;
        for(int i
        {
            int c
            if(!ne
            p = ne
        }
        cnt[p] +=
    }
    int query(stri
    {
        int p = 0,
        for(int i
        {
            int c
            if(!ne
            p = ne
            ans+=c
        }
        return ans
    }
} tree;
int main()
{
    ios::sync_with
    int n,m;
    cin>>n>>m;
    for(int i=1; i
    {
        string s;
        cin>>s;
        tree.inser
    }
    for(int i=1; i
    {
        string s;
        cin>>s;
        cout<<tree
    }
    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;
struct trietree
{
    struct trie_node
    {
        int isKey = 0; //标志，0：不是字符串结尾，>0：记录出现次数
        trie_node* child[26] = {nullptr}; //指向子节点指针
    };
    trie_node* root=new trie_node;
    //插入字符串：
    void insert(string s, int len)
    {
        trie_node* p = root;
        for(int i = 0; i < len; i++)
        {
            int n = s[i] - 'a';
            if(!p->child[n]) //没有对应子节点，创建
            {
                trie_node* q =new trie_node;
                p->child[n] = q;
            }
            p = p->child[n];
        }
        p->isKey += 1;//字符串结尾标志位置为1
    }
    //检索字符串
    int query(string s, int len)
    {
        trie_node* p = root;
        int ans=0;
        for(int i = 0; i < len; i++)
        {
            int n = s[i] - 'a';
            if(!p->child[n])return ans;
            p = p->child[n];
            ans+=p->isKey;
        }
        return ans;
    }
} tree;
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int n,m;
    cin>>n>>m;
    for(int i=1; i<=n; i++)
    {
        string s;
        cin>>s;
        tree.insert(s,s.length());
    }
    for(int i=1; i<=m; i++)
    {
        string s;
        cin>>s;
        cout<<tree.query(s,s.length())<<"\n";
    }
    return 0;
}
```



回 顾 总 结







COOLMAKER
酷哥创客 人工智能教育

酷哥创客

感谢您的观看

